Visualization

Course Outline

1

Simulation Data Visualization

Built-in Package

2

Plotting

- Plot Generation
- Multi-physics Plotting

2

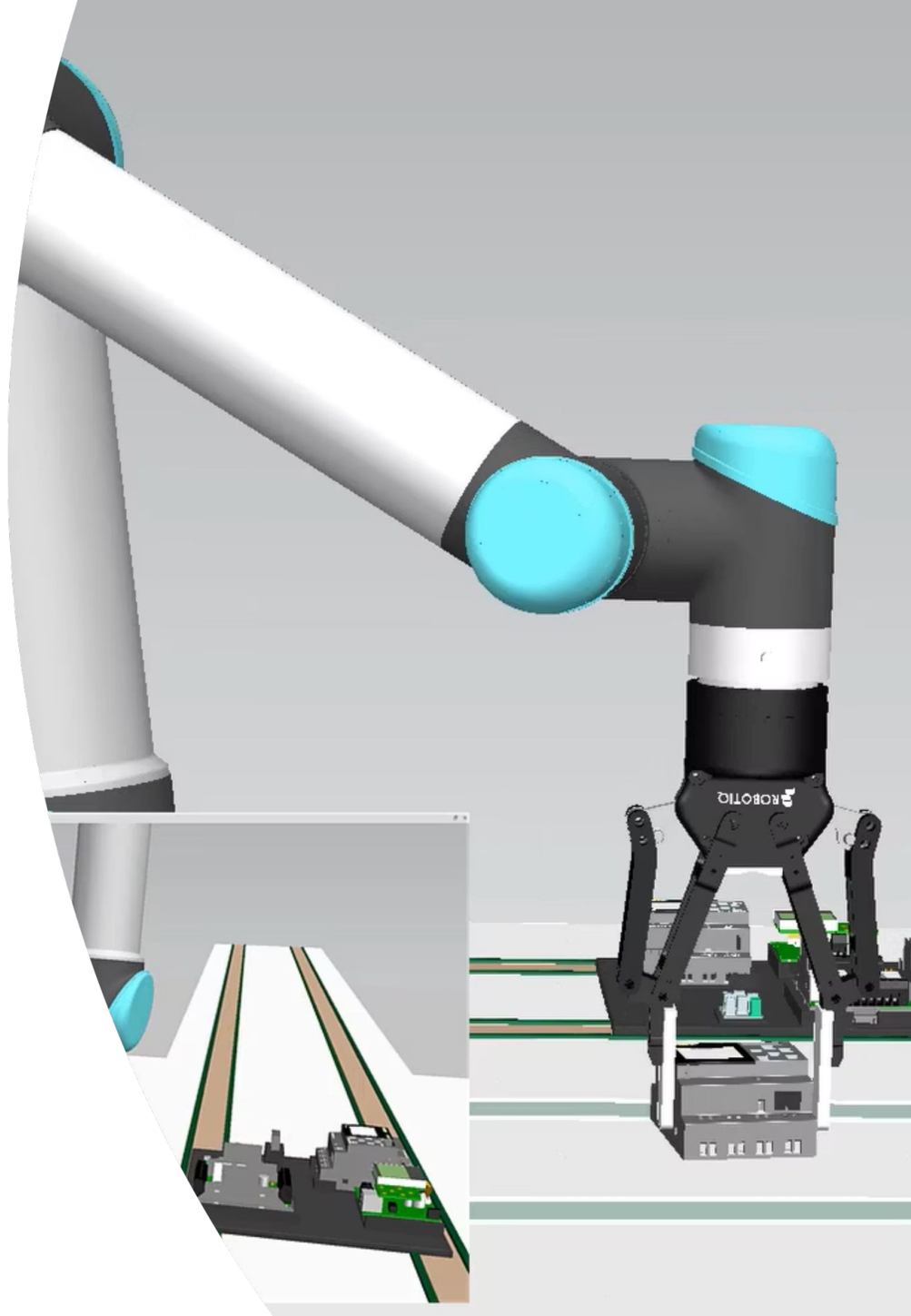
Path Tracing

- Curve Tracking
- Colormap

What You Will Learn in This Course

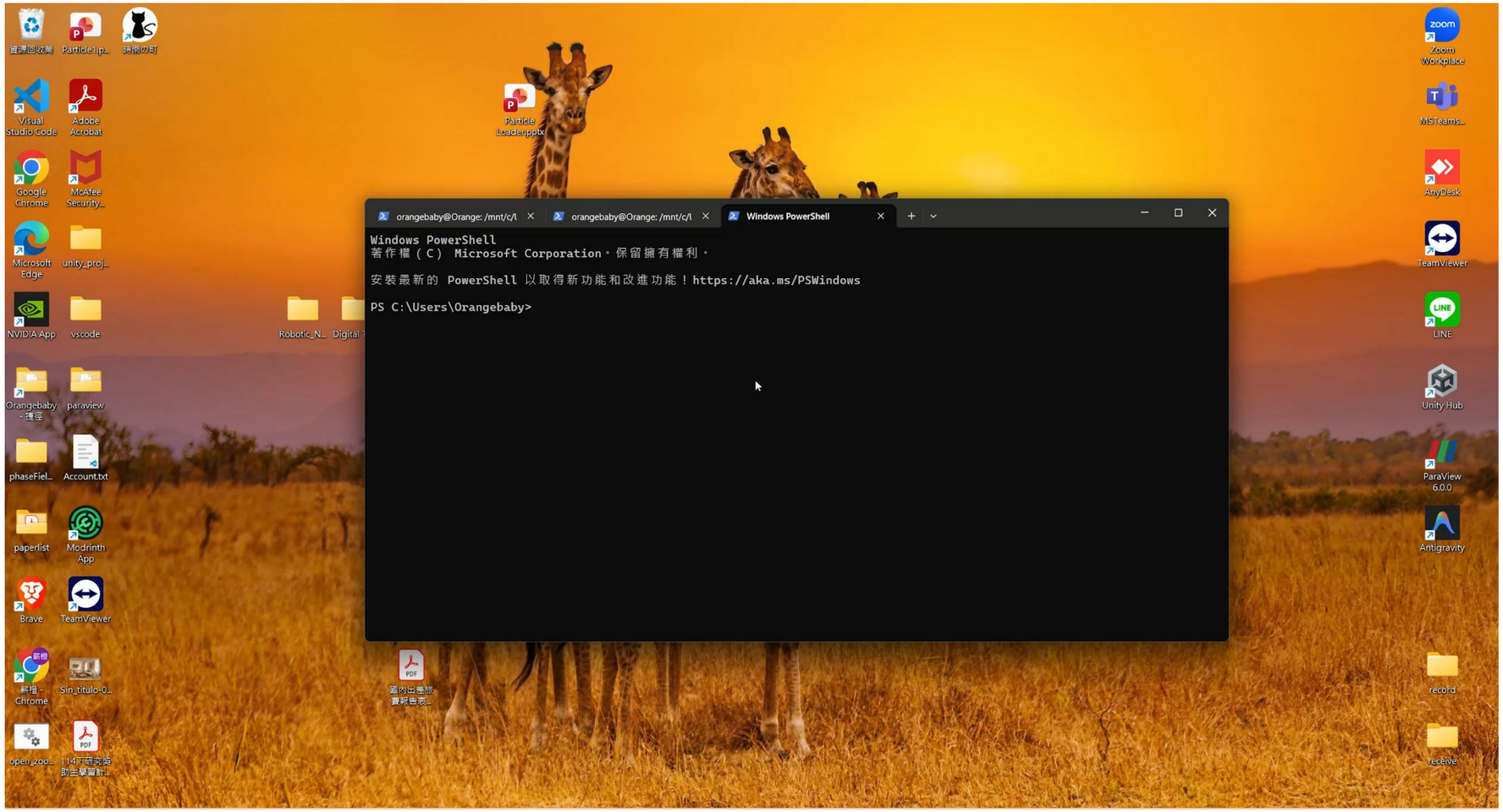
1. Open the build-in simulation data visualizer
2. Extract data and plot customized physical quantities over time
3. Track motion and visualize trajectories with heatmap color mapping

Simulation Data Visualization



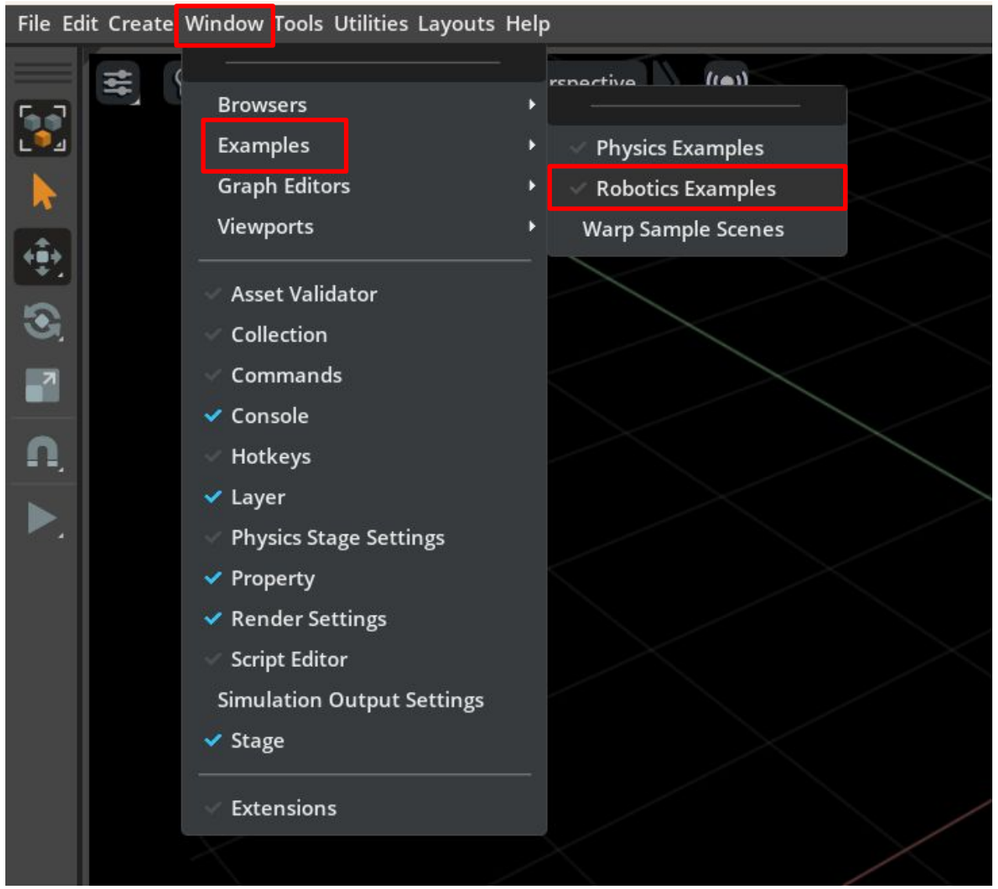
Launch application

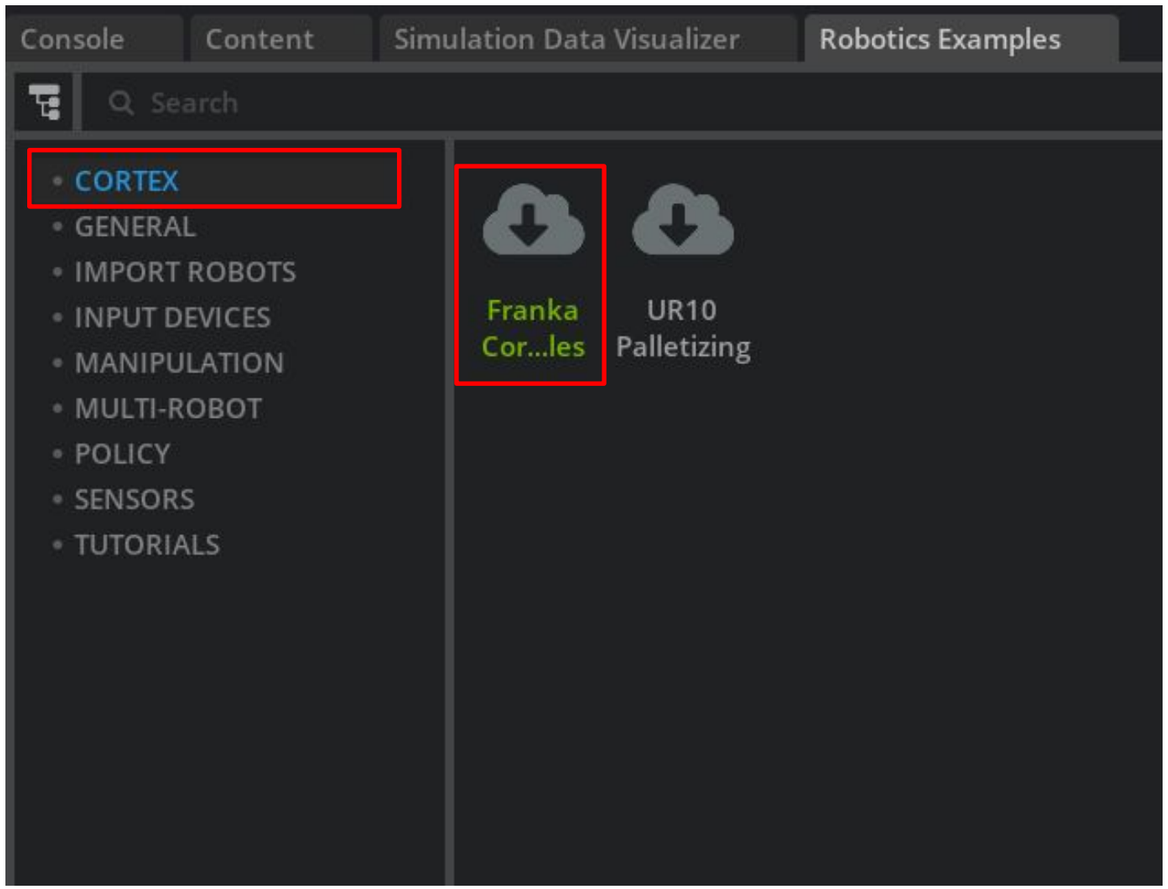
- Navigate to your project folder:
  [your path]/isaac-sim
- Run: **.\isaac-sim.bat**



Open Franka Cortex Example

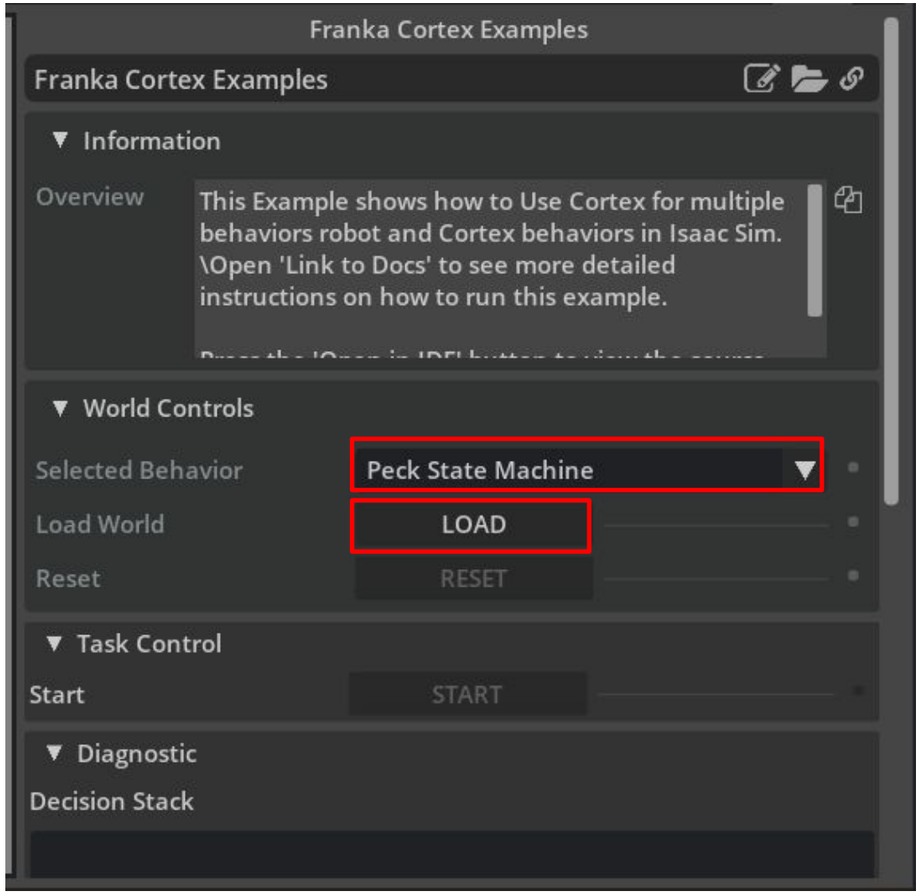
- Open the Robotic examples list, go to **Window > Examples > Robotics Examples**
- Select **CORTEX > Franka Cortex Example** at content view

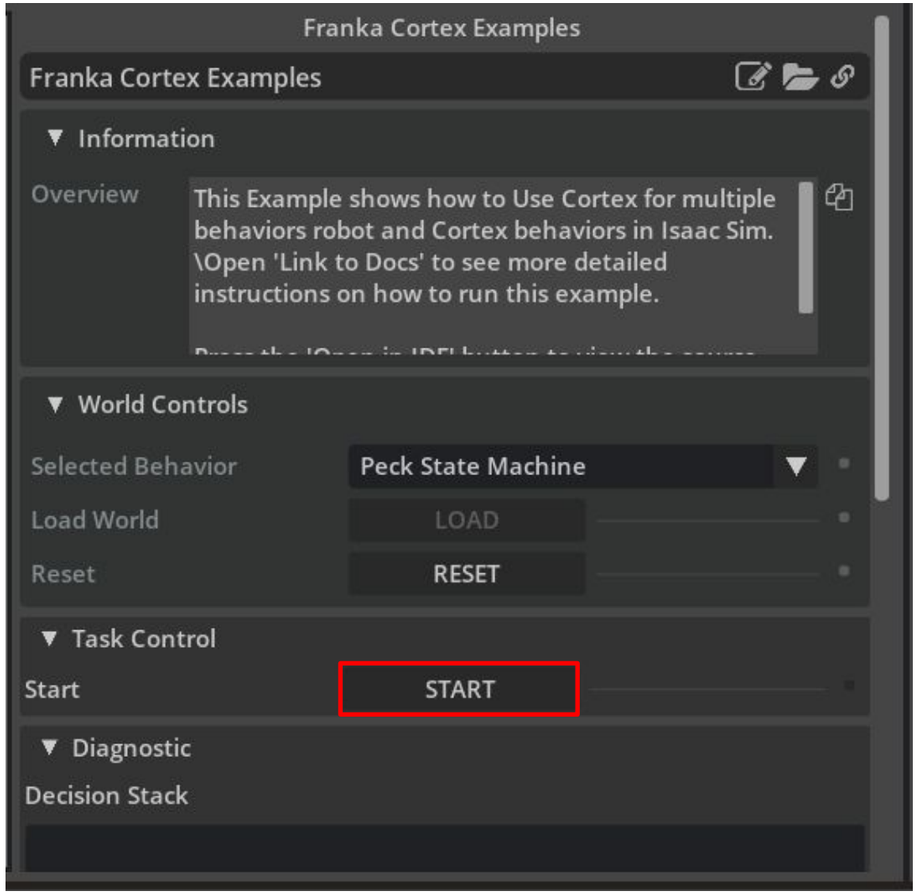




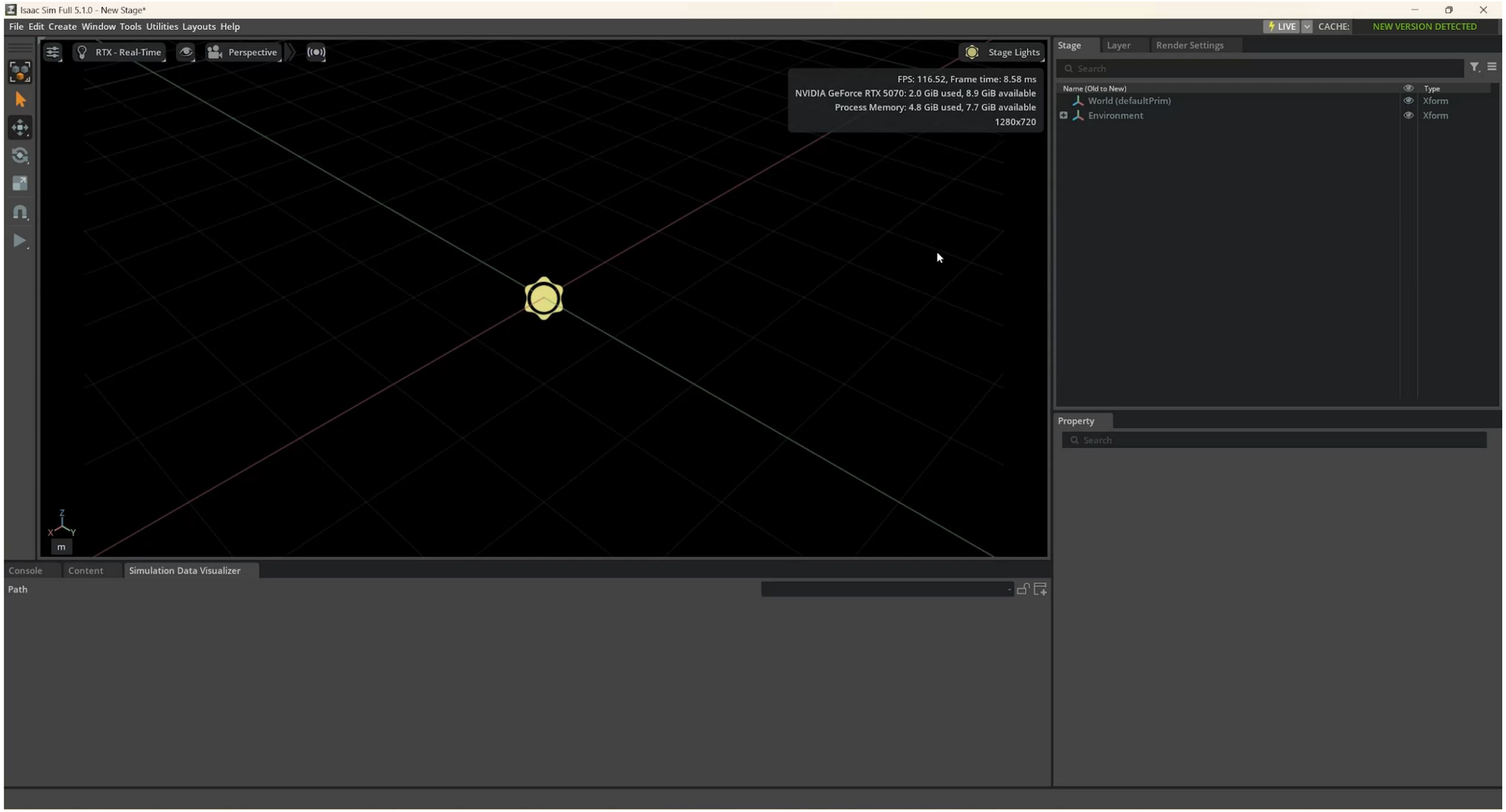
Open Franka Cortex Example

- In **World Controls > Selected Behavier**, select **Peck State Machine > LOAD**
- Select **Task Control > START** to start the task simulation on franka



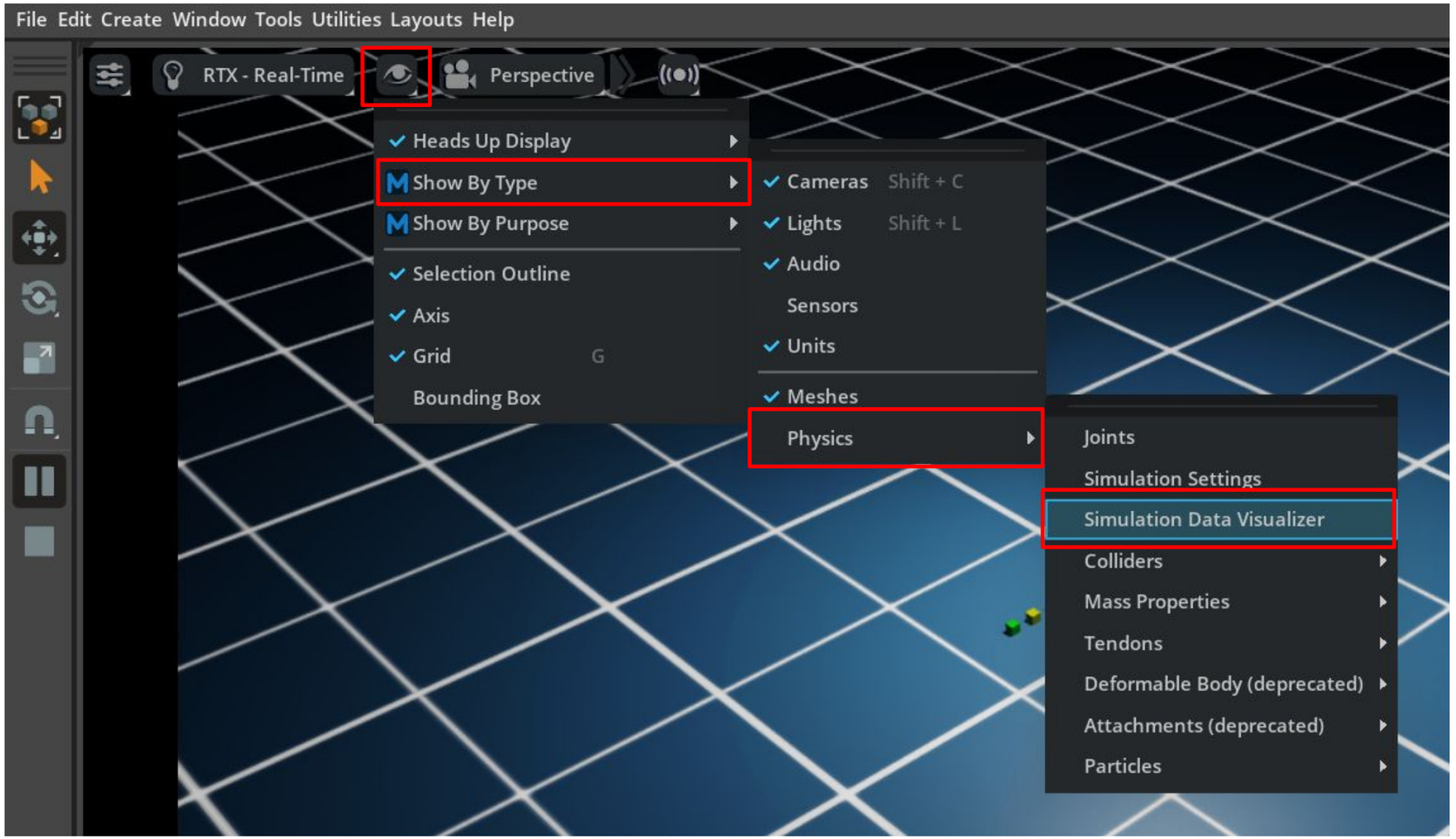


Open Franka Cortex Example (Demo)



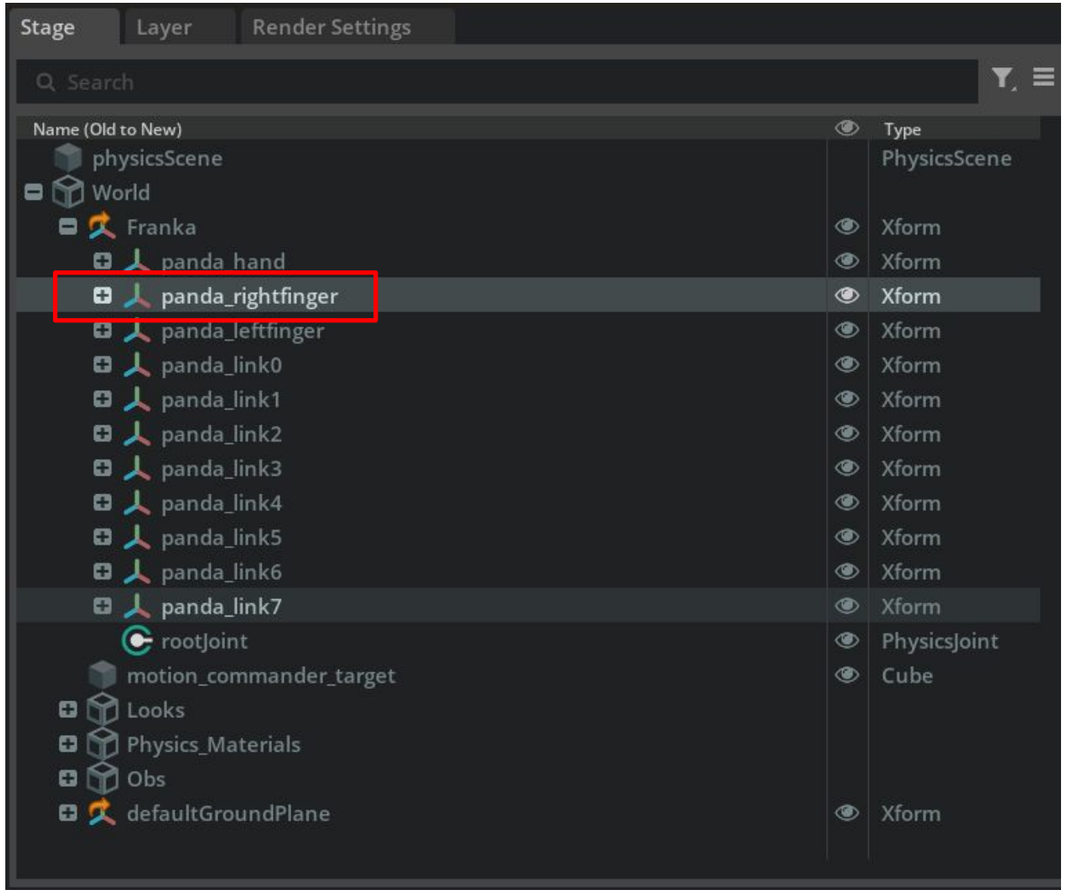
Open Simulation Data Visualization

- In the viewport, click **Eyes > Show By Type > Physics > Simulation Data Visualizer**

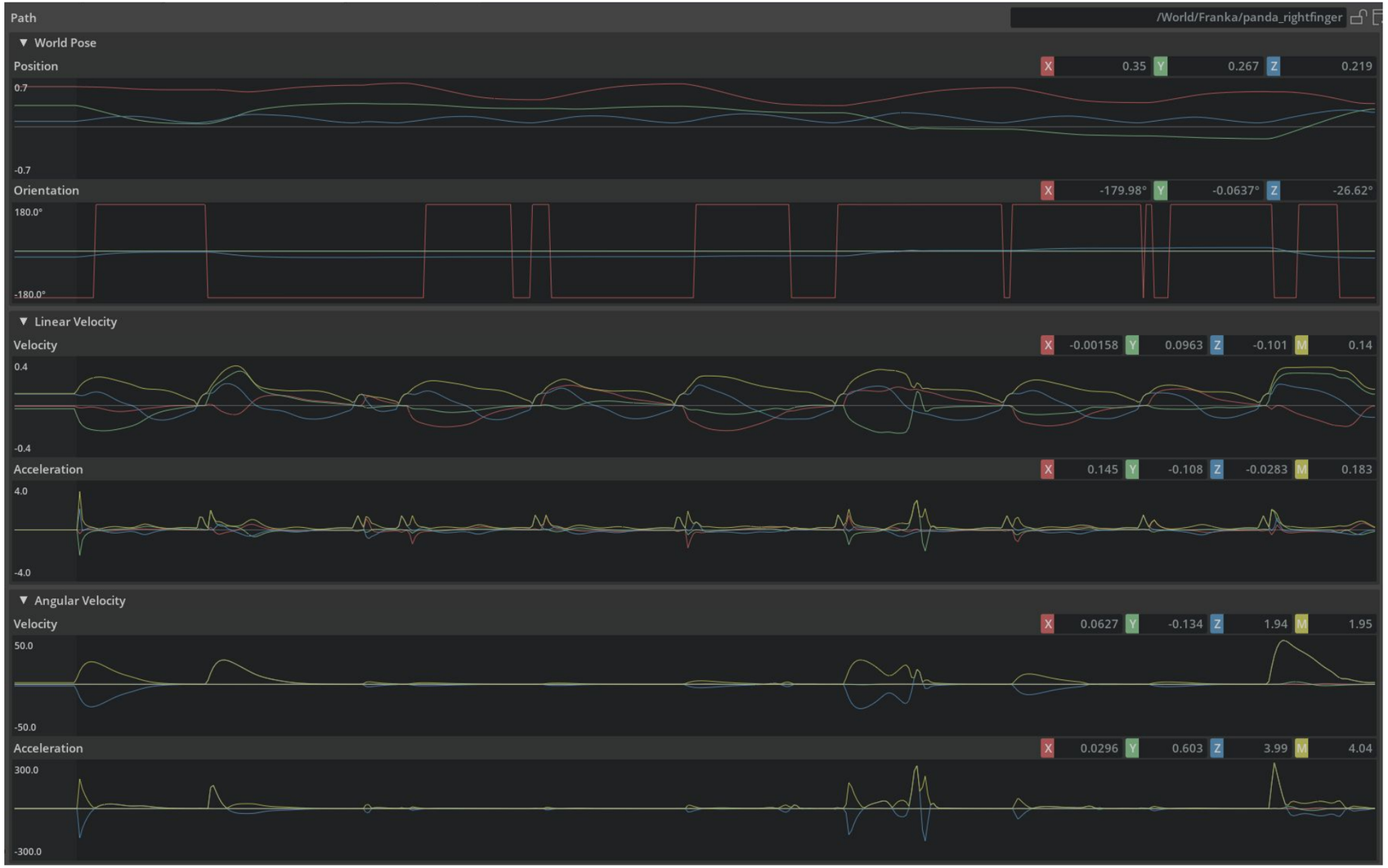


Open Simulation Data Visualization

- In stage tree, Select **World > Franka > panda_rightfinger**
- You can see the physics parameter plotting on content view (Simulation Data Visualizer)



Open Simulation Data Visualization



Position
Orientation

Linear velocity
Linear acceleration

Angular velocity
Angular acceleration

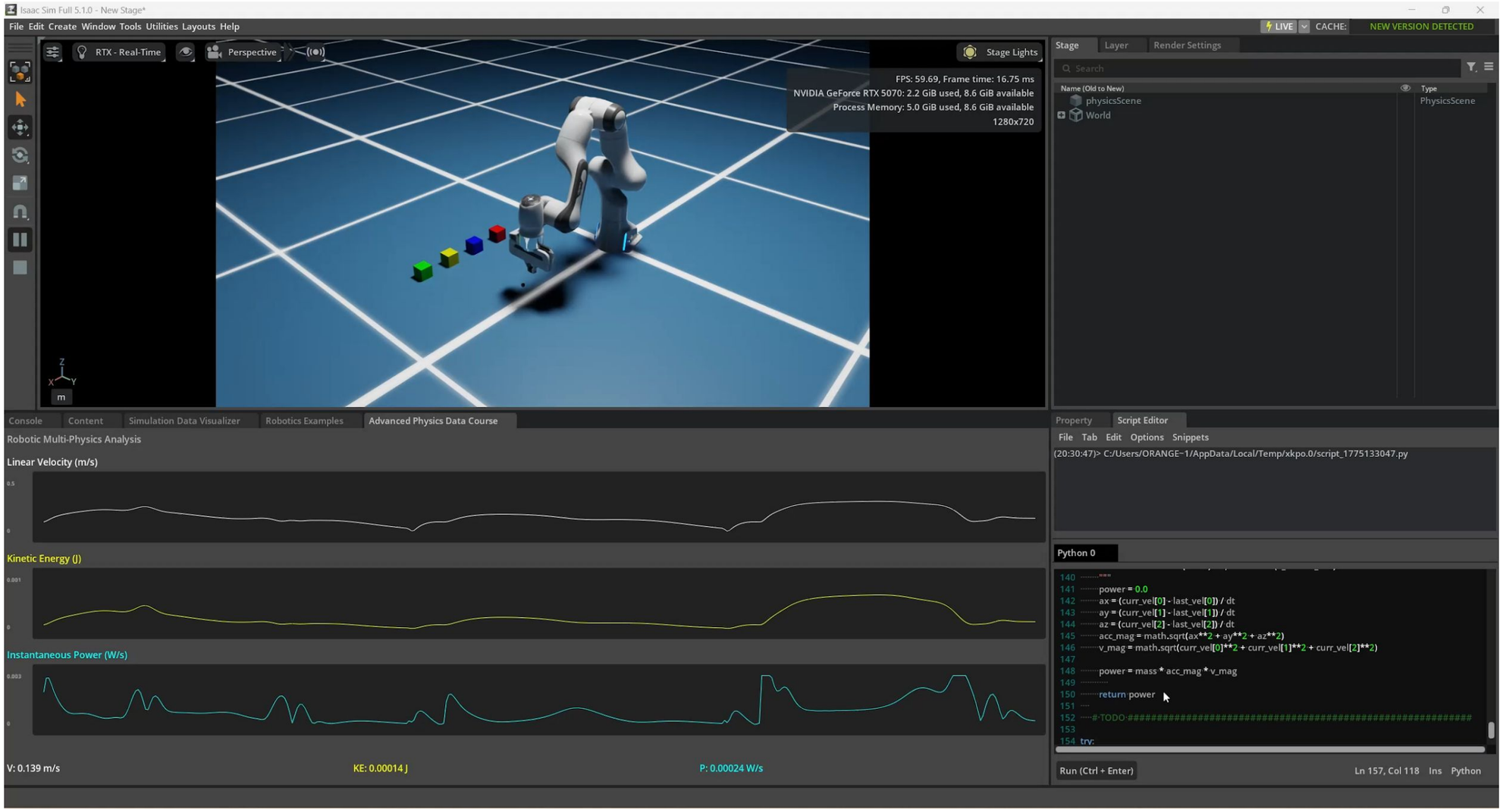
Visualization Examples (Hand-on)

- Go to google drive download the hand-on script:
  - plotting.py
  - tracing.py

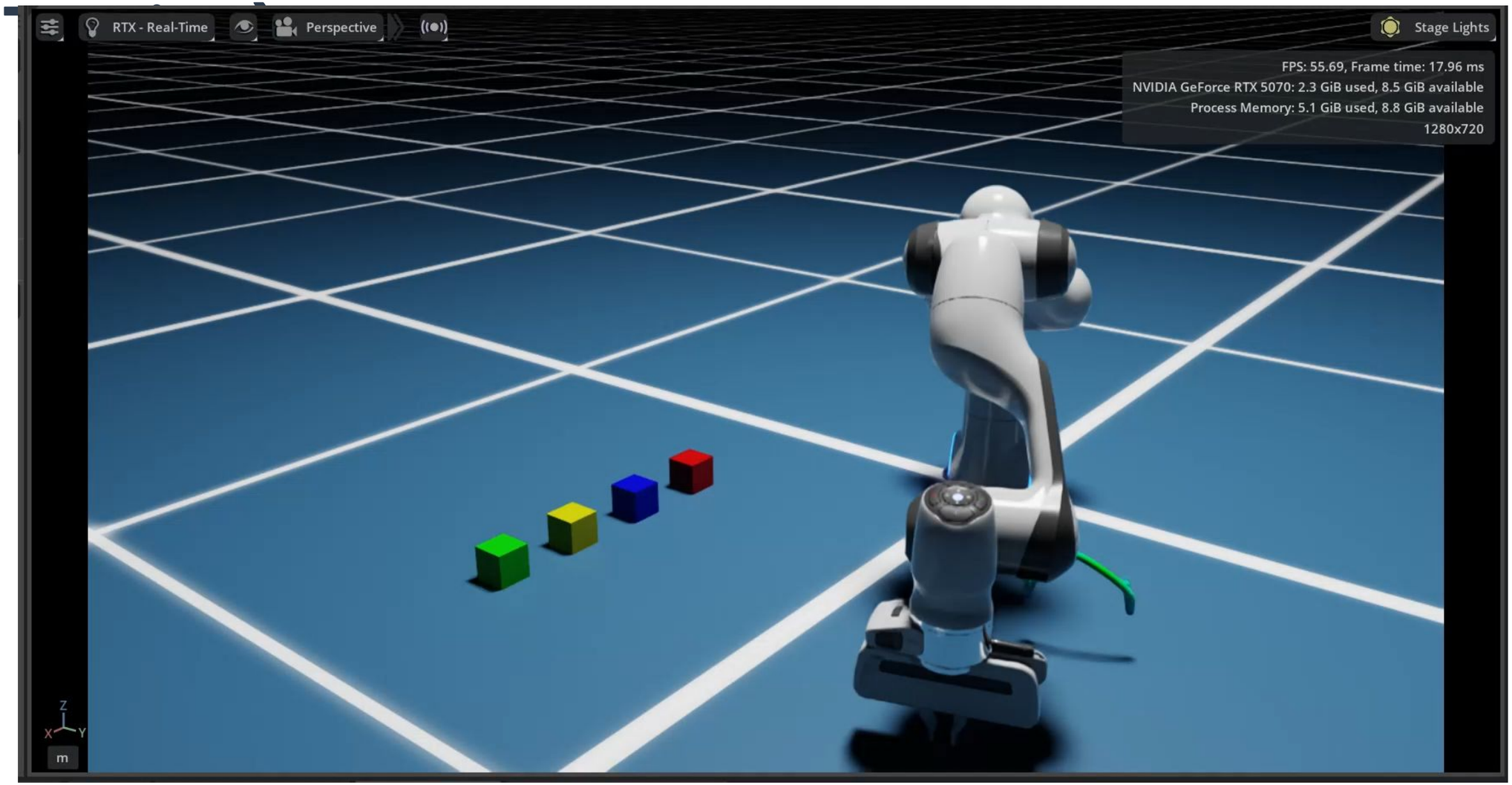
- Follow the instructions to complete each work:
  - Plotting (Part 2)
  - Path Tracing (Part 3)

- There are total 4 TODO section in these two script, you need to implement each function and make it work correct.

Visualization Examples (Plotting)

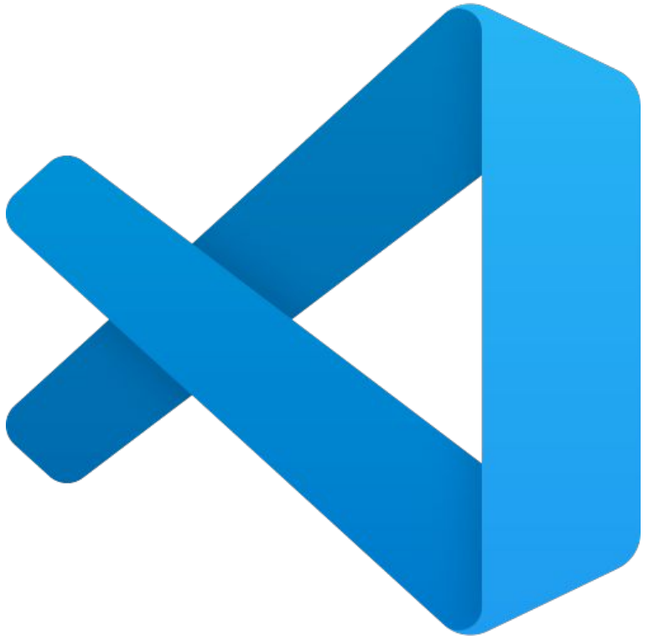


Visualization Examples (Path



How to editor scripts

Visual Studio

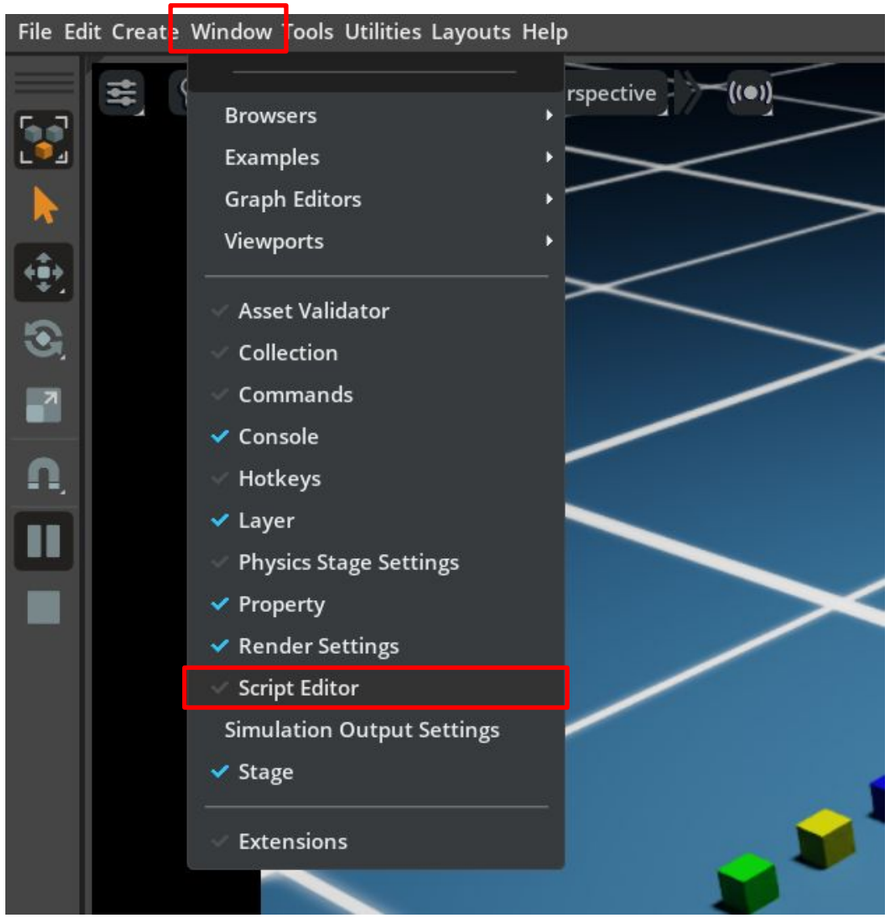


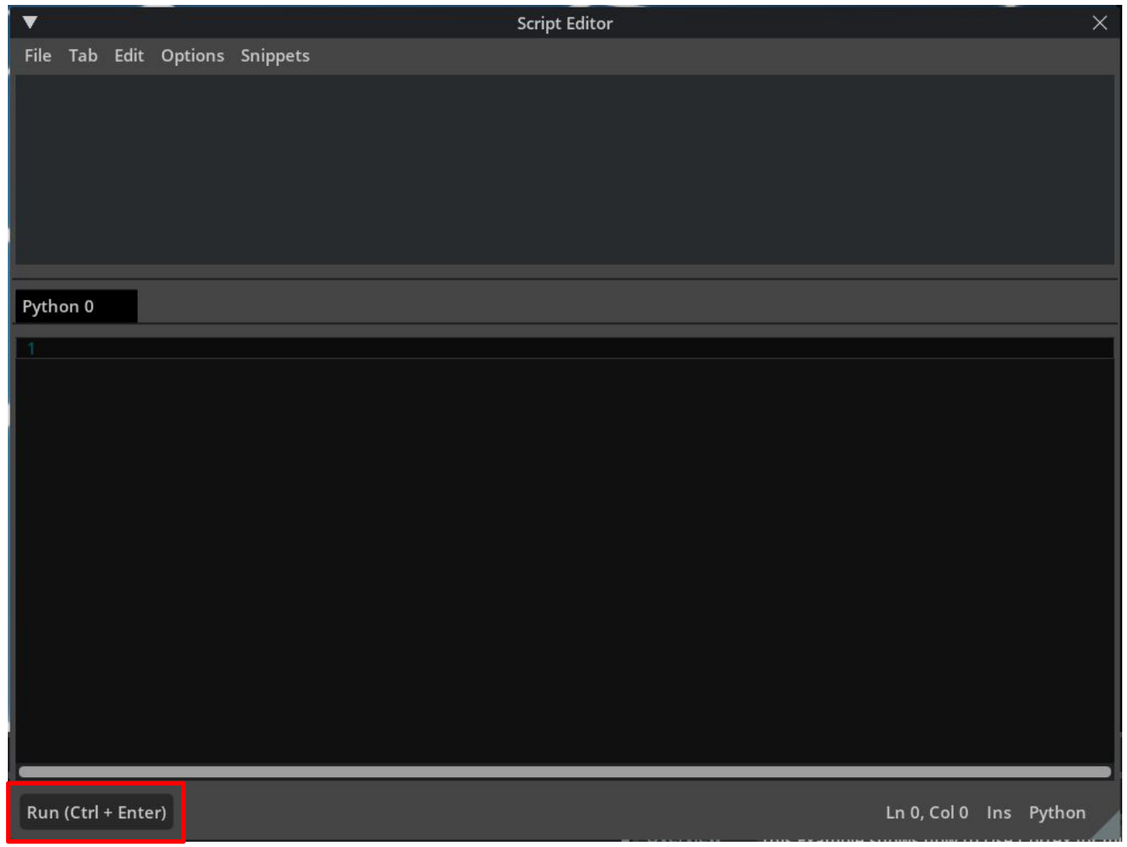
VS code

Antigravity

How to execute the scripts

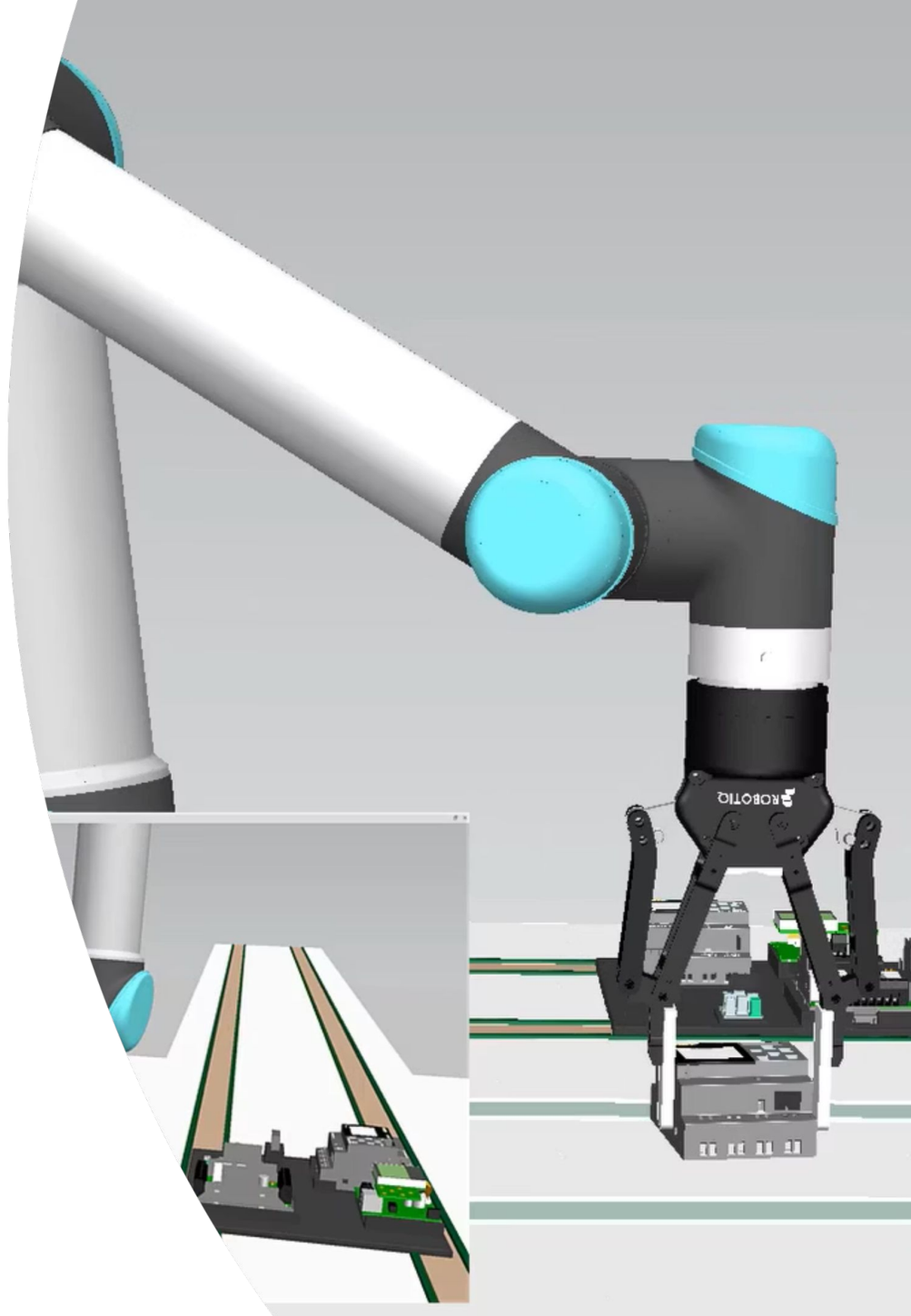
- Use **Window > Script** Editor to run python code directly.





Plotting

Plot Generation



Code Overview

```
def create_ui_window():
    ...

def create_plot_section(parent, label, max_val, color):
    ...

def on_update(e: carb.events.IEvent):
    ...

def calculate_velocity_magnitude(velocity_vector):
    ...

def calculate_kinetic_energy(v_magnitude, mass):
    ...

def calculate_instantaneous_power(curr_vel, last_vel, dt, mass):
    ...
```

Global UI Design and Layout

Plot section

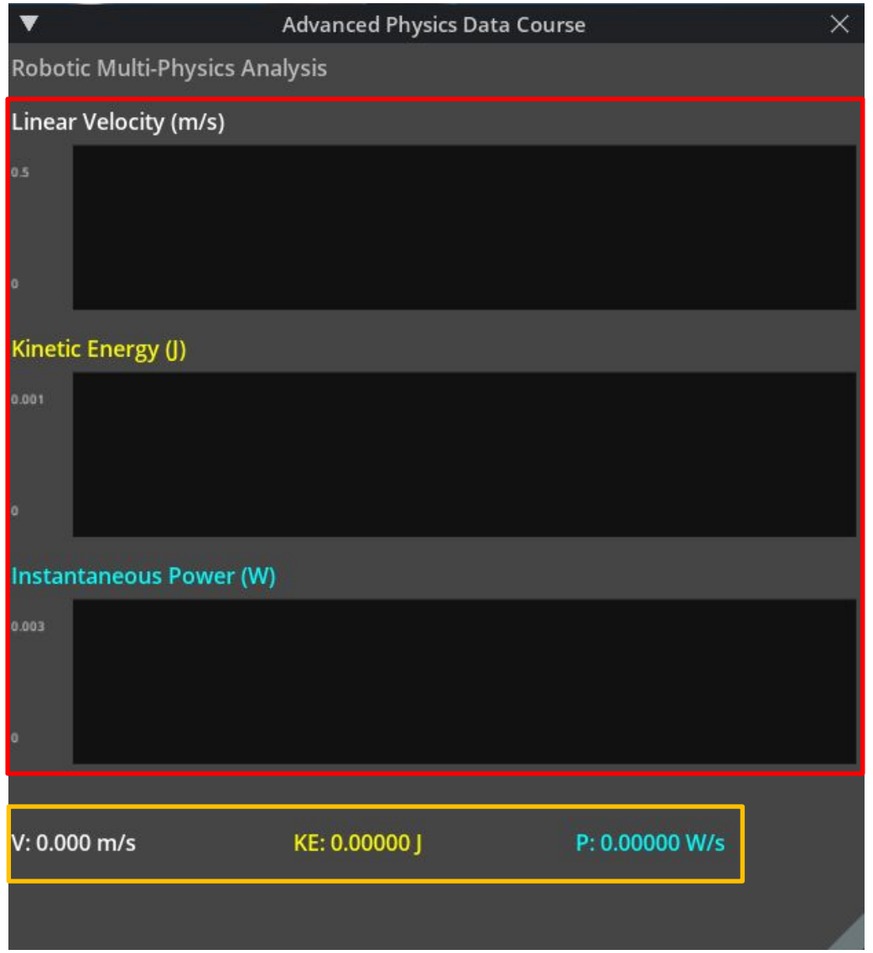
Update Logic (TODO1)

Physics function (TODO2)

Global UI

```
def create_ui_window():
    ...
    # --- 1. Velocity Plot ---
    vel_plot = create_plot_section(ui.VStack(), "Linear Velocity (m/s)", V_MAX, 0xFFFFFFFF)

    # --- 2. Kinetic Energy Plot ---
    ke_plot = create_plot_section(ui.VStack(), "Kinetic Energy (J)", KE_MAX, 0xFF00FFFF)

    # --- 3. Power Plot ---
    p_plot = create_plot_section(ui.VStack(), "Instantaneous Power (W/s)", P_MAX,
0xFFFFFF00)


    # --- Data Overview ---
    ui.Spacer(height=5)
    with ui.HStack(height=30):
        v_label = ui.Label("V: 0.00 m/s", style={"color": 0xFFFFFFFF, "font_size": 16})
        ke_label = ui.Label("KE: 0.00000 J", style={"color": 0xFF00FFFF, "font_size": 16})
        p_label = ui.Label("P: 0.000 W/s", style={"color": 0xFFFFFF00, "font_size": 16})
```

```
def create_plot_section(parent, label, max_val, color):
    ...
```

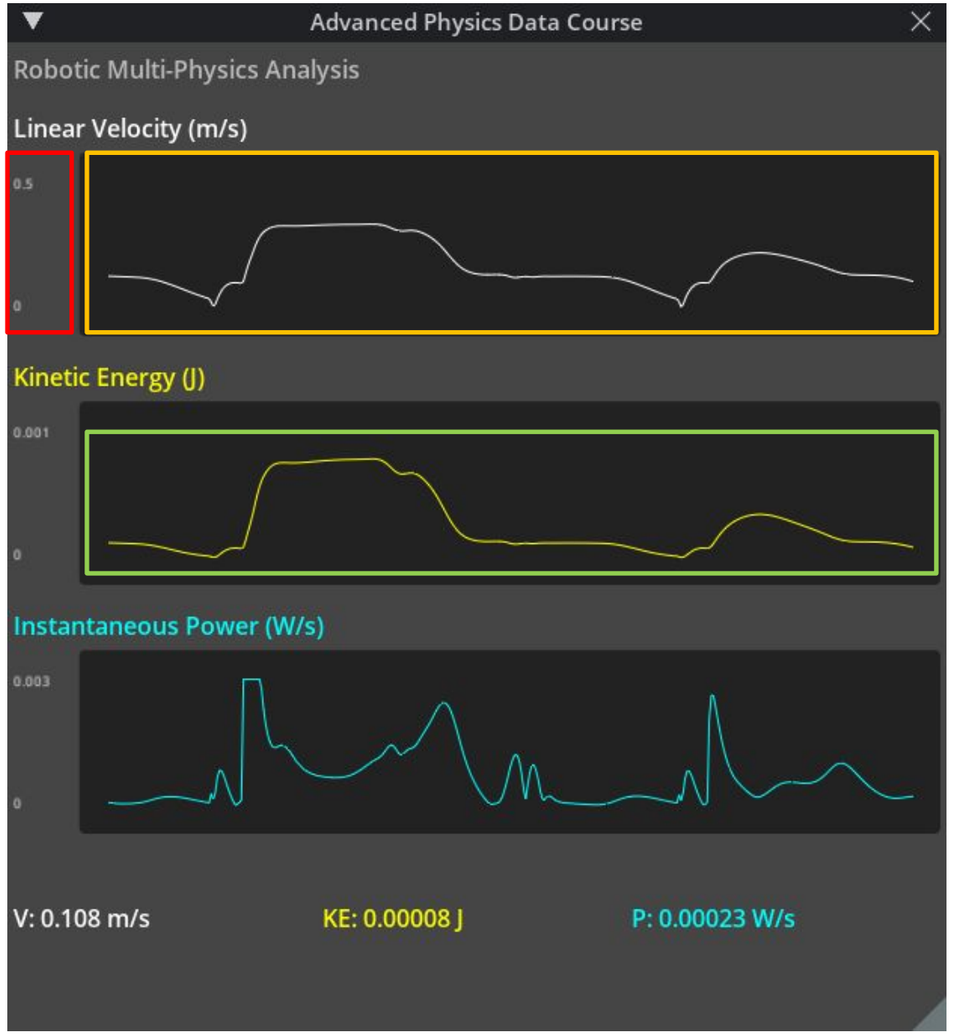


Overview

- Set vertical coordinate
- Set background color
- Set plotting color

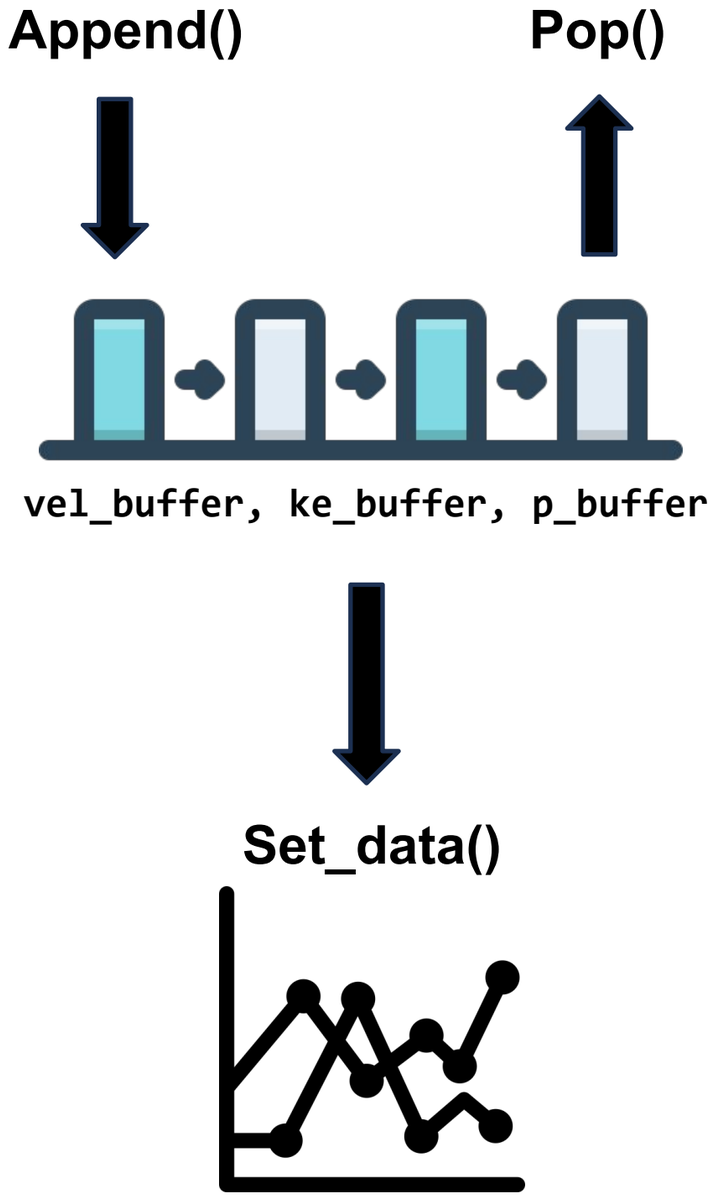
```
def create_plot_section(parent, label, max_val, color):
    """Create a plot section and return the plot object"""
    with parent:
        with ui.VStack(height=120):
            ui.Label(label, height=15, style={"font_size": 16, "color": color})
            ui.Spacer(height=5)
            with ui.HStack():

                with ui.VStack(width=35):
                    ui.Label(str(max_val), style={"font_size": 9});
                    ui.Spacer(); ui.Label("0", style={"font_size": 9})

                with ui.ZStack():
                    ui.Rectangle(style={"background_color": 0xFF111111,
                                        "border_color": 0xFF444444, "border_width": 1})
```

```
                plot = ui.Plot(ui.Type.LINE, 0.0, max_val,
                               style={"color": color, "line_width": 1.8})
```



Plot Section

```
def on_update(e: carb.events.IEvent):
    ...

    vel_state = dc.get_rigid_body_linear_velocity(handle)
    if vel_state:
        curr_vel_vec = [vel_state.x, vel_state.y, vel_state.z]

        # call helper functions
        v_mag = calculate_velocity_magnitude(curr_vel_vec)
        ke = calculate_kinetic_energy(v_mag, mass)
        power = calculate_instantaneous_power(curr_vel_vec, last_vel_vec, dt, mass)

        # update Buffers
        self.vel_buffer.pop(0)
        self.vel_buffer.append(v_mag)
        ...

        # update plots
        vel_plot.set_data(*vel_buffer)
        ...

        # update data labels
        v_label.text = f"V: {v_mag:.3f} m/s"
        ke_label.text = f"KE: {ke:.5f} J"
        p_label.text = f"P: {power:.5f} W/s"

        last_vel_vec = curr_vel_vec
```



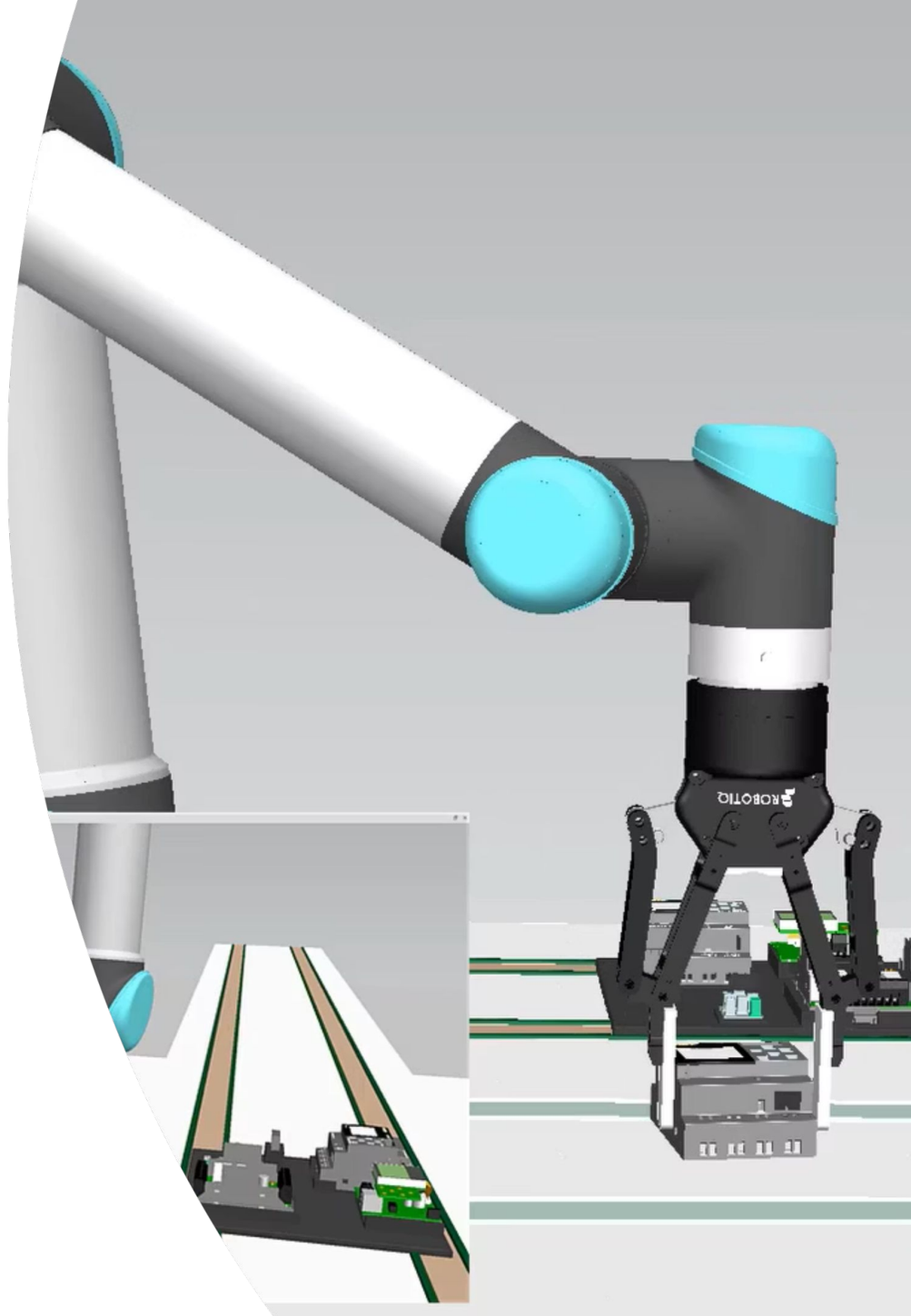
TODO Section 1

- Maintain the data buffer for plots updating.
- Use **set_val()** to pass the data sequence to each buffer.

```
vel_state = self.dc.get_rigid_body_linear_velocity(self.handle)
if vel_state:
    curr_vel_vec = np.array([vel_state.x, vel_state.y, vel_state.z])

    # call helper functions
    v_mag = self.calculate_velocity_magnitude(curr_vel_vec)
    ke = self.calculate_kinetic_energy(v_mag, self.mass)
    power = self.calculate_instantaneous_power(curr_vel_vec, self.last_vel_vec, dt, self.mass)

    #! Do not modify the code outside of the TODO sections.
    #! Update data buffers and plots with the calculated values from the helper functions
    # TODO1 ##############################################################
    # update Buffers


    # update plots


    # TODO1 ##############################################################

    # update data labels
    self.v_label.text = f"V: {v_mag:.3f} m/s"
    self.ke_label.text = f"KE: {ke:.5f} J"
    self.p_label.text = f"P: {power:.5f} W"
```

Plotting

Multi-physics Plotting



Multi-Physics Plotting

- Velocity Magnitude: $\|v\| = \sqrt{v_x^2 + v_y^2 + v_z^2}$
- Kinematic Energy: $Ke = \frac{1}{2}mv^2$
- Instantaneous Power: $P = \mathrm{m} \times a \times v$ where $\mathrm{a} = \frac{v_{curr} - v_{last}}{dt}$

$v = [v_x, v_y, v_z]$: velocity vector
m: mass of object
v_{curr}: current velocity
v_{last}: previous velocity
dt: delta time

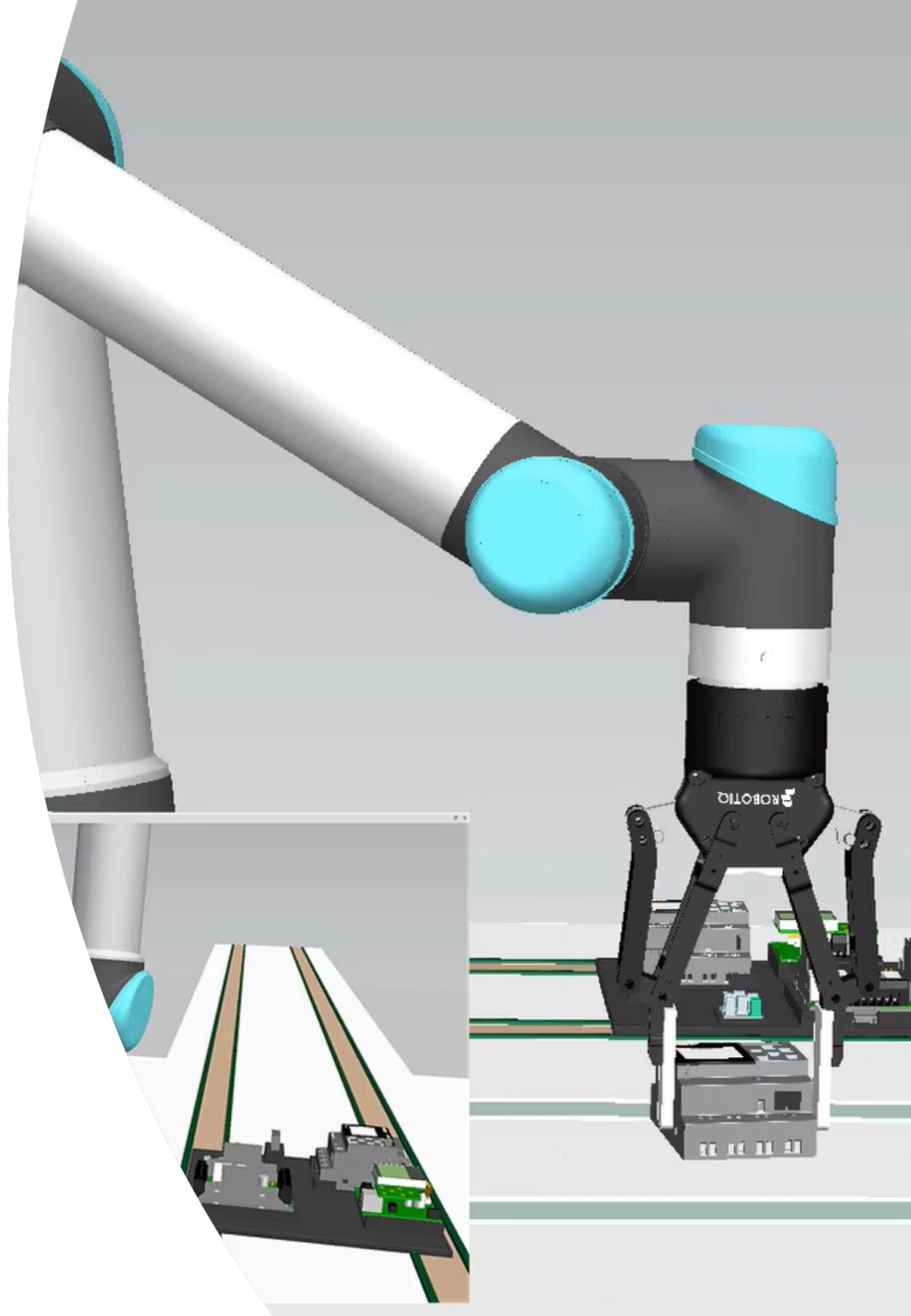
TODO Section 2

- Complete the three physics calculation.

```
#! Do not modify the code outside of the TODO sections.
#! You need to implement the three functions below to complete the course exercises.
# TODO2 ##############################################################

def calculate_velocity_magnitude(self, velocity_vector):
    """
    Task 1: calculate velocity magnitude
    """
    v_mag = 0.0
    return v_mag


def calculate_kinetic_energy(self, v_magnitude, mass):
    """
    Task 2: calculate kinetic energy
    """
    ke = 0.0
    return ke


def calculate_instantaneous_power(self, curr_vel, last_vel, dt, mass):
    """
    Task 3: calculate instantaneous power
    """
    power = 0.0
```

Path Tracing

Curve Tracking



Overview

```
# Get current target position
def get_target_position():
    ...


# Create one colored segment
def create_segment(p0, p1, color, seg_idx):
    ...


# Update callback
def update_trajectory_curve_time(event):
    ...


def speed_to_color(speed, vmin=MIN_SPEED, vmax=MAX_SPEED):
    ...
```

Get object position (p_x, p_y, p_z)

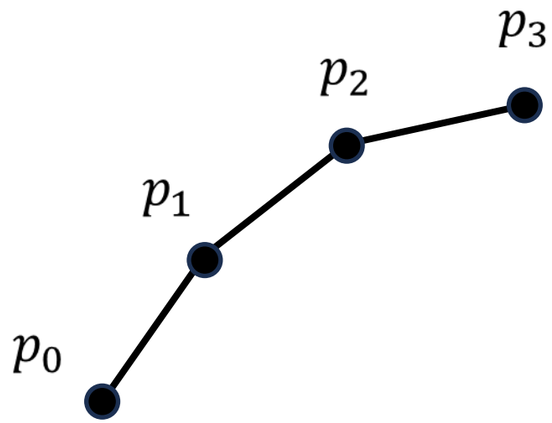
Create a linear segment

Update segments buffer

Color mapping function

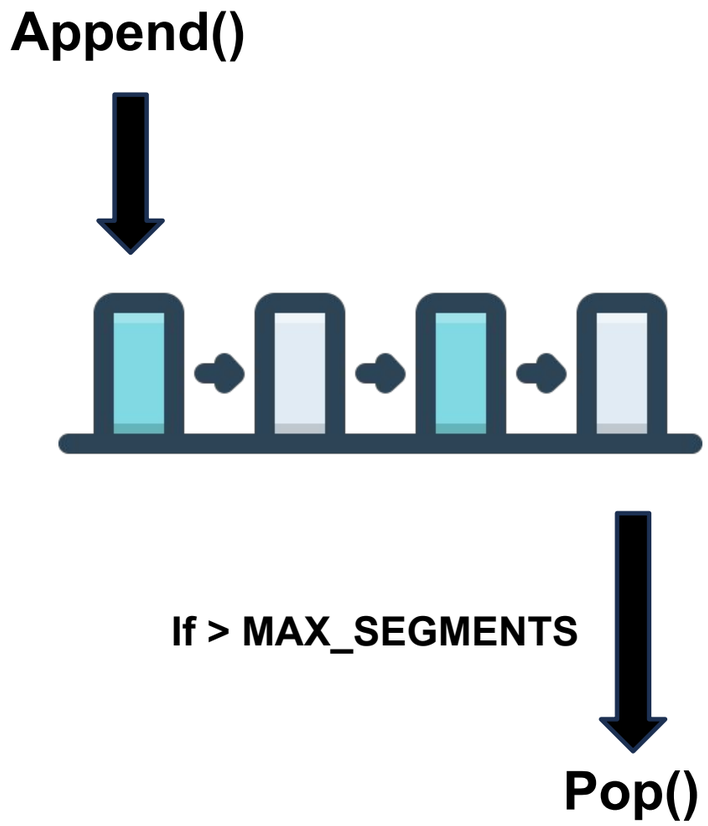
Create a Linear Segment

```python
def create_segment(p0, p1, color, seg_idx):
    seg_path = f"{SEGMENTS_ROOT}/seg_{seg_idx}"

    curve = UsdGeom.BasisCurves.Define(stage, seg_path)
    curve.CreateTypeAttr(UsdGeom.Tokens.linear)
    curve.CreateBasisAttr(UsdGeom.Tokens.bspline)
    curve.GetPointsAttr().Set([p0, p1])
    curve.GetCurveVertexCountsAttr().Set([2])
    curve.GetWidthsAttr().Set([CURVE_WIDTH, CURVE_WIDTH])
    curve.GetDisplayColorAttr().Set([color])

    return seg_path


def update_trajectory_curve_time(event):
    ...

    # create new segment
    seg_path = create_segment(last_point, new_point, color, segment_count)
    segment_paths.append(seg_path)
    segment_count += 1

    ...
```



Update Segments Buffer

```python
def update_trajectory_curve_time(event):
    global last_point, segment_paths, segment_count, time_acc

    ...
    sample_dt = time_acc

    ...
    dist = (new_point - last_point).GetLength()

    if dist > 1e-4:

        # calculate speed
        speed = dist / sample_dt

        # map to color
        color = speed_to_color(speed)

        # create new segment
        seg_path = create_segment(last_point, new_point, color, segment_count)
        segment_paths.append(seg_path)
        segment_count += 1

        # Maintain a fixed maximum number of segments
        if len(segment_paths) > MAX_SEGMENTS:
            stage.RemovePrim(segment_paths.pop(0))

        # update last point
        last_point = new_point
```



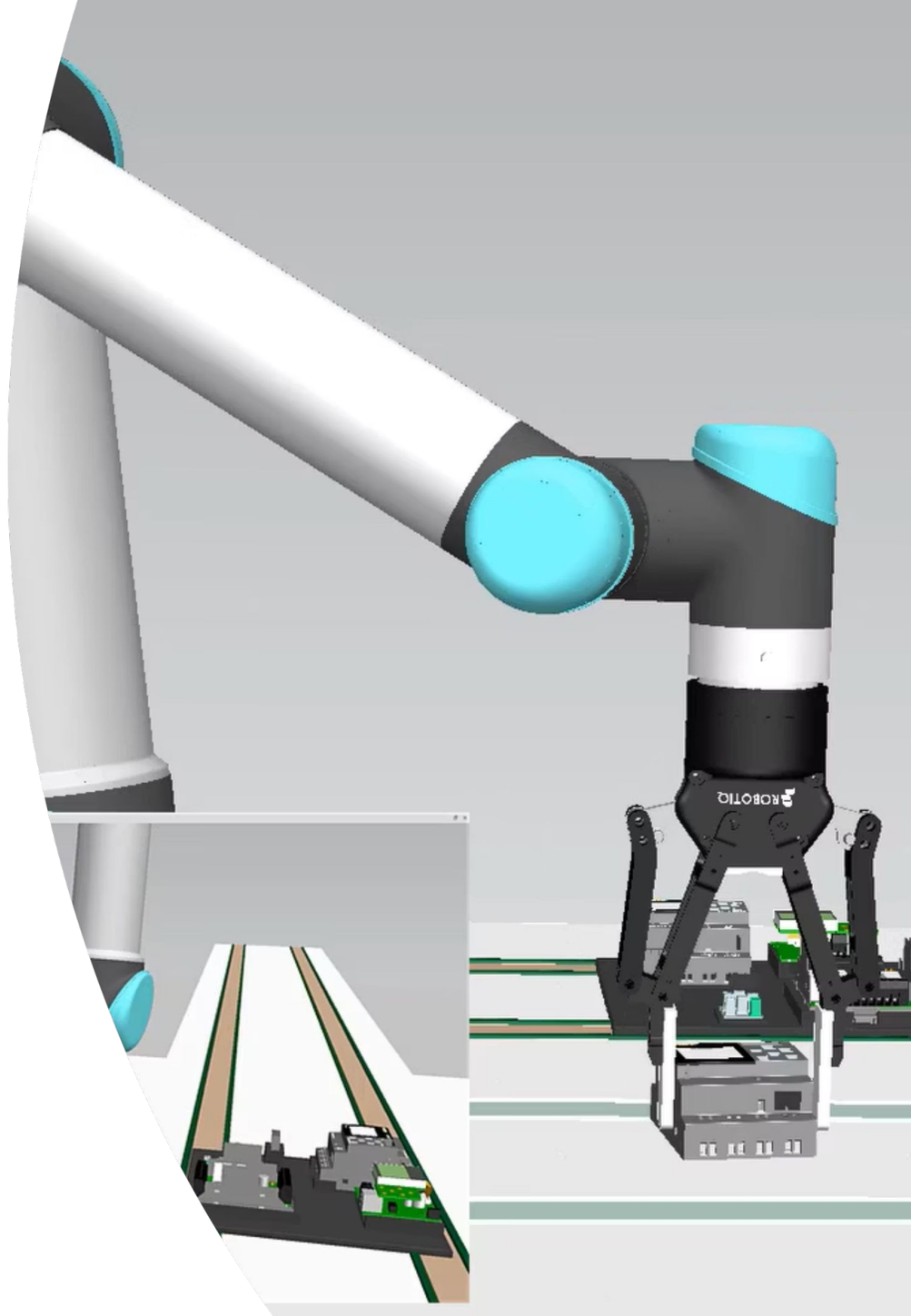
TODO Section 3

- Calculate the speed and then pass to colormap function.
- Maintain a segment buffer to keep the **maximum number of segments ≤ buffer_size**

```
# Gate new position and calculate displacement
new_point = get_target_position()
dist = (new_point - last_point).GetLength()

if dist > 1e-4:

    #! Do not modify the code outside of the TODO scetions.
    #! Calculate speed and map to color, create new segment,
    #! maintain a fixed maximum number of segments, and update last point
    # TODO3 ##########################################################

    # calculate speed
    speed = 0.0

    # map to color
    color = speed_to_color(speed)



    # create new segment
    # Maintain a fixed maximum number of segments
    # update last point
```

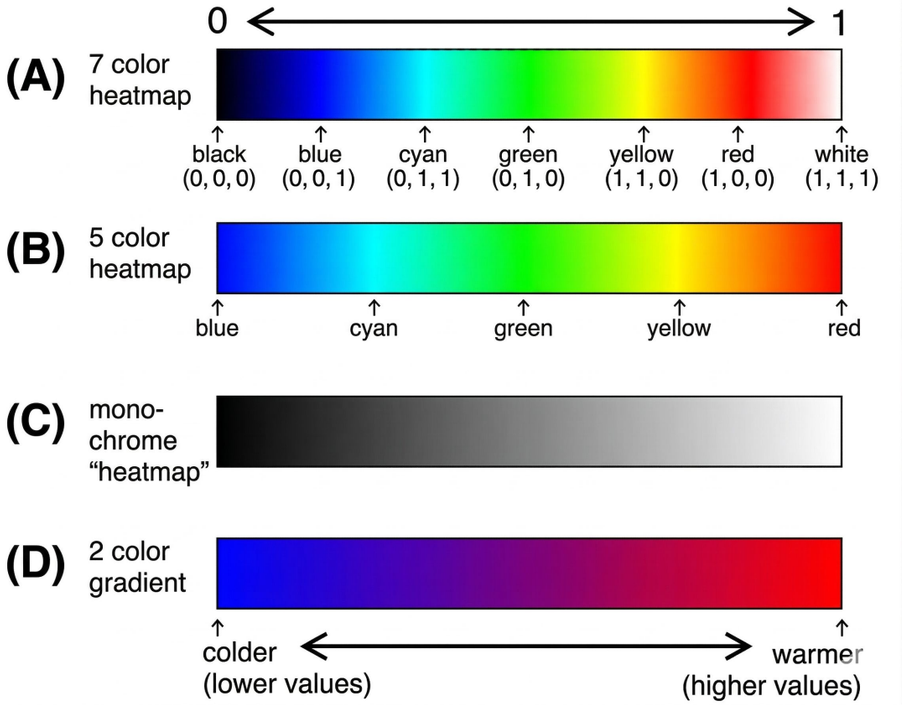
Path Tracing

Colormap



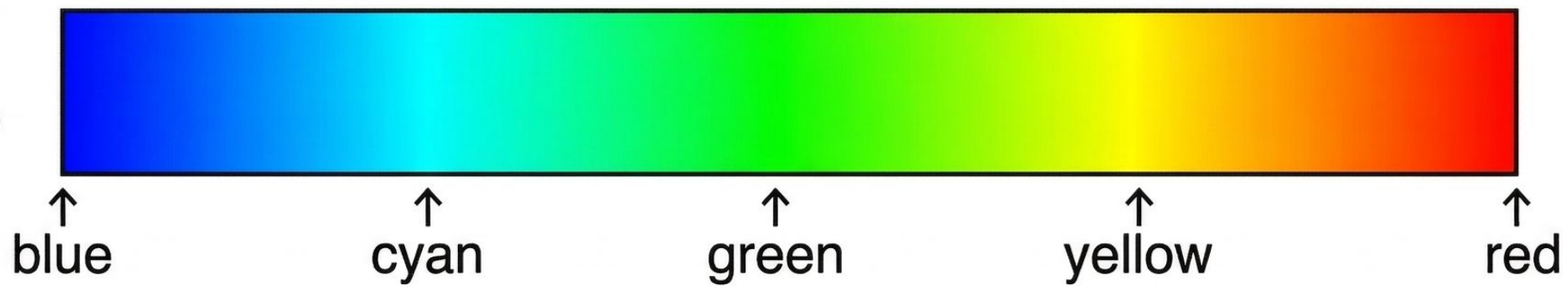
Colormap

- Map numerical values to colors
  - Input: scalar values (e.g., speed, temperature, pressure)
  - Output: RGB colors
- Colormaps are widely used to visualize scalar data across science, graphics, and data analysis.



Translation Guidance

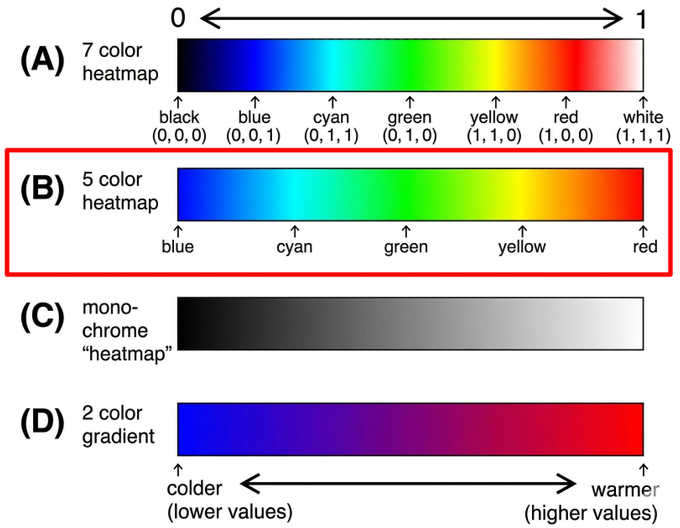
- Step1: Normalize [Min, Max] into [0,1]
- Step2: Partition the colormap into 4 segment
  - [blue, cyan], [cyan, green], [green, yellow], [yellow, red]
- Step3: Map value to color segment by segment
  - [0.0, 0.25], [0.25, 0.5], [0.5, 0.75], [0.75, 1.0]
- Step4: Implement the interpolation function for each segment.



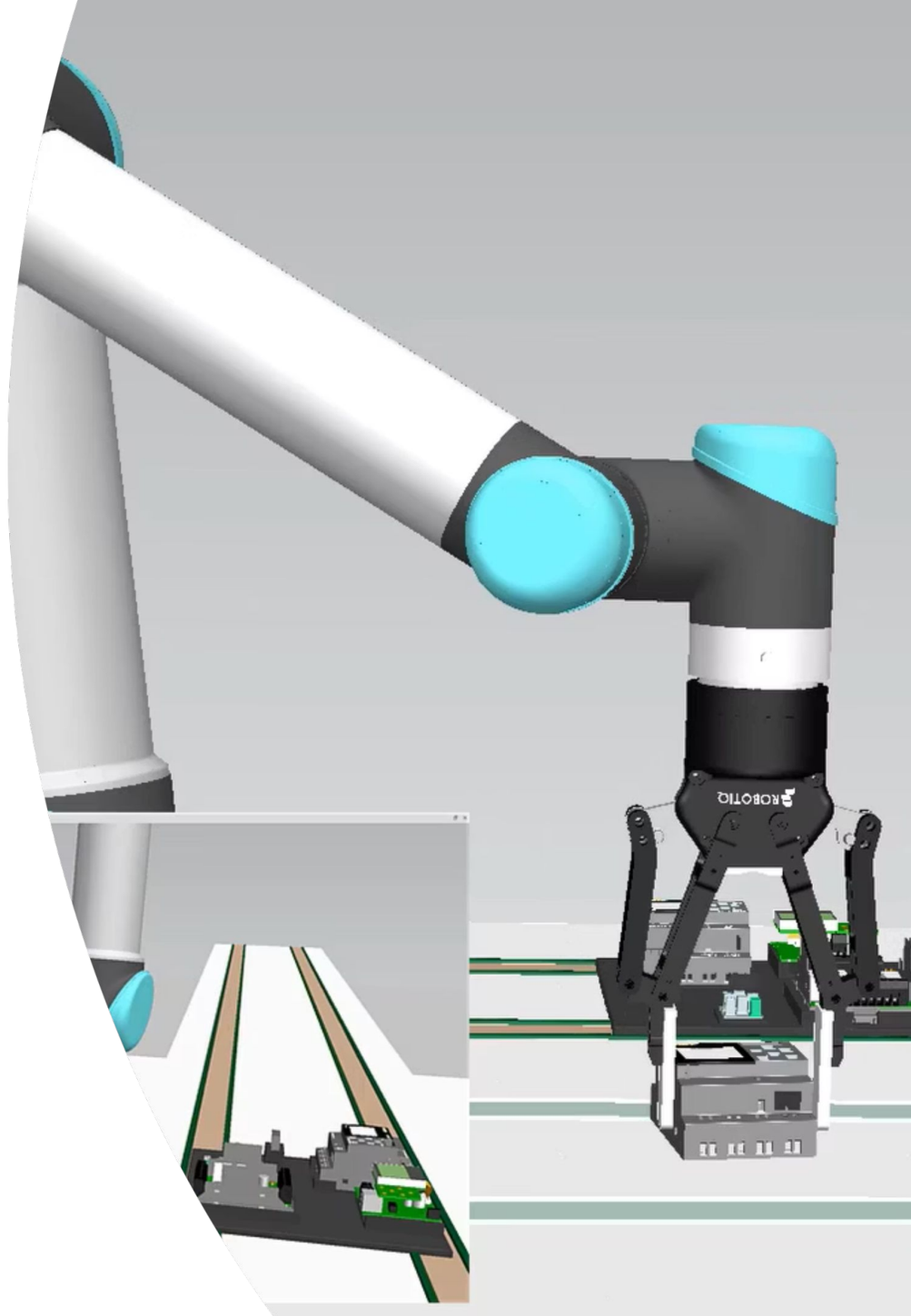
TODO Section 4

- Implement a function to map the speed (range from MIN_SPEED to MAX_SPEED) to **5-color-heatmap**.

```
#! Do not modify the code outside of the TODO scetions.
#! Implementation of the color mapping function for 5 color heatmap (blue, cyan, green, yellow, red)
# TODO4 ####################################
def speed_to_color(speed, vmin=MIN_SPEED, vmax=MAX_SPEED):
    t = 0.0 if vmax <= vmin else (speed - vmin) / (vmax - vmin)
    t = max(0.0, min(1.0, t))


    colors = [
        (0.0, 0.0, 1.0),  # blue
        (0.0, 1.0, 1.0),  # cyan
        (0.0, 1.0, 0.0),  # green
        (1.0, 1.0, 0.0),  # yellow
        (1.0, 0.0, 0.0),  # red
    ]


    r = 1.0; g = 0.0; b = 0.0
    return (r, g, b)
# TODO4 ####################################
```



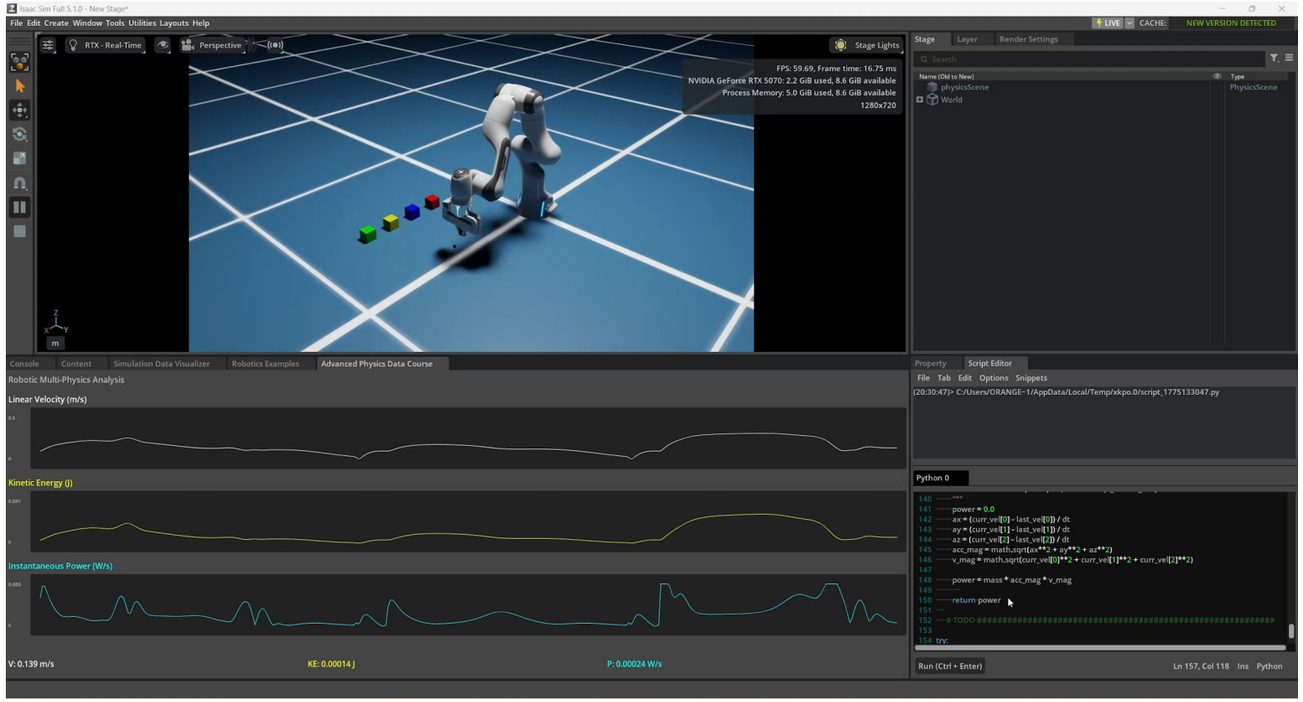
Q&A

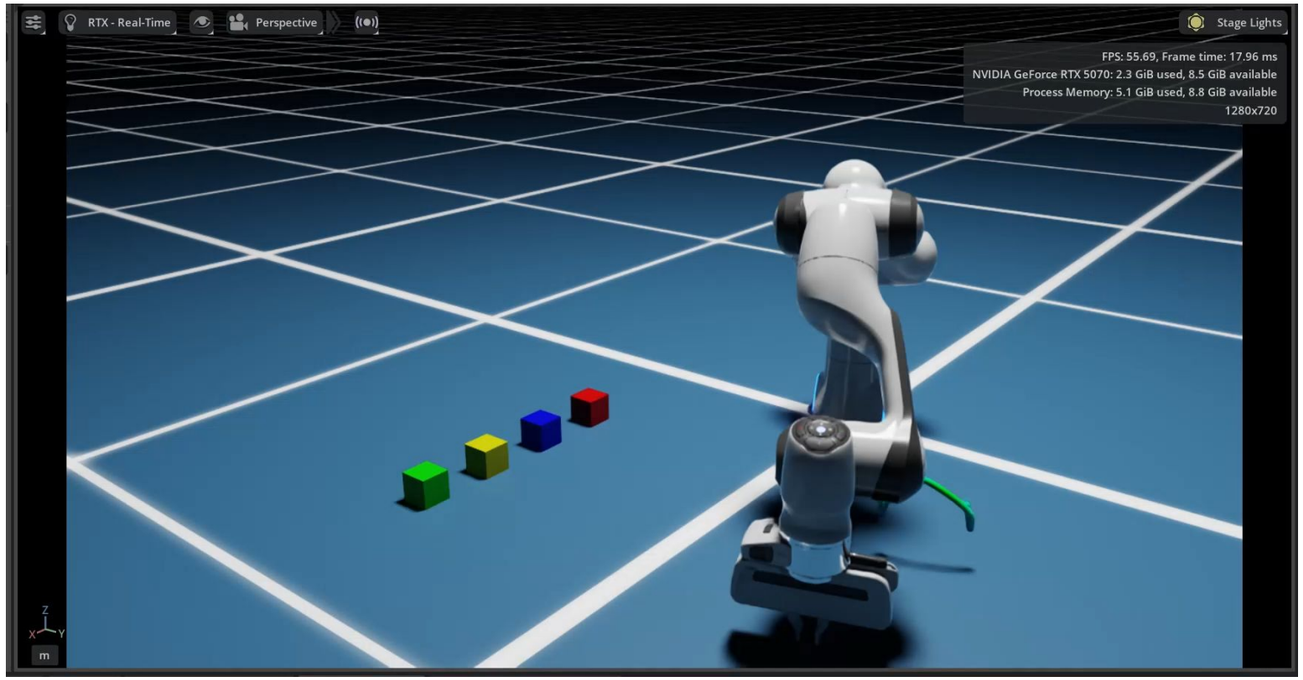


Lab Task

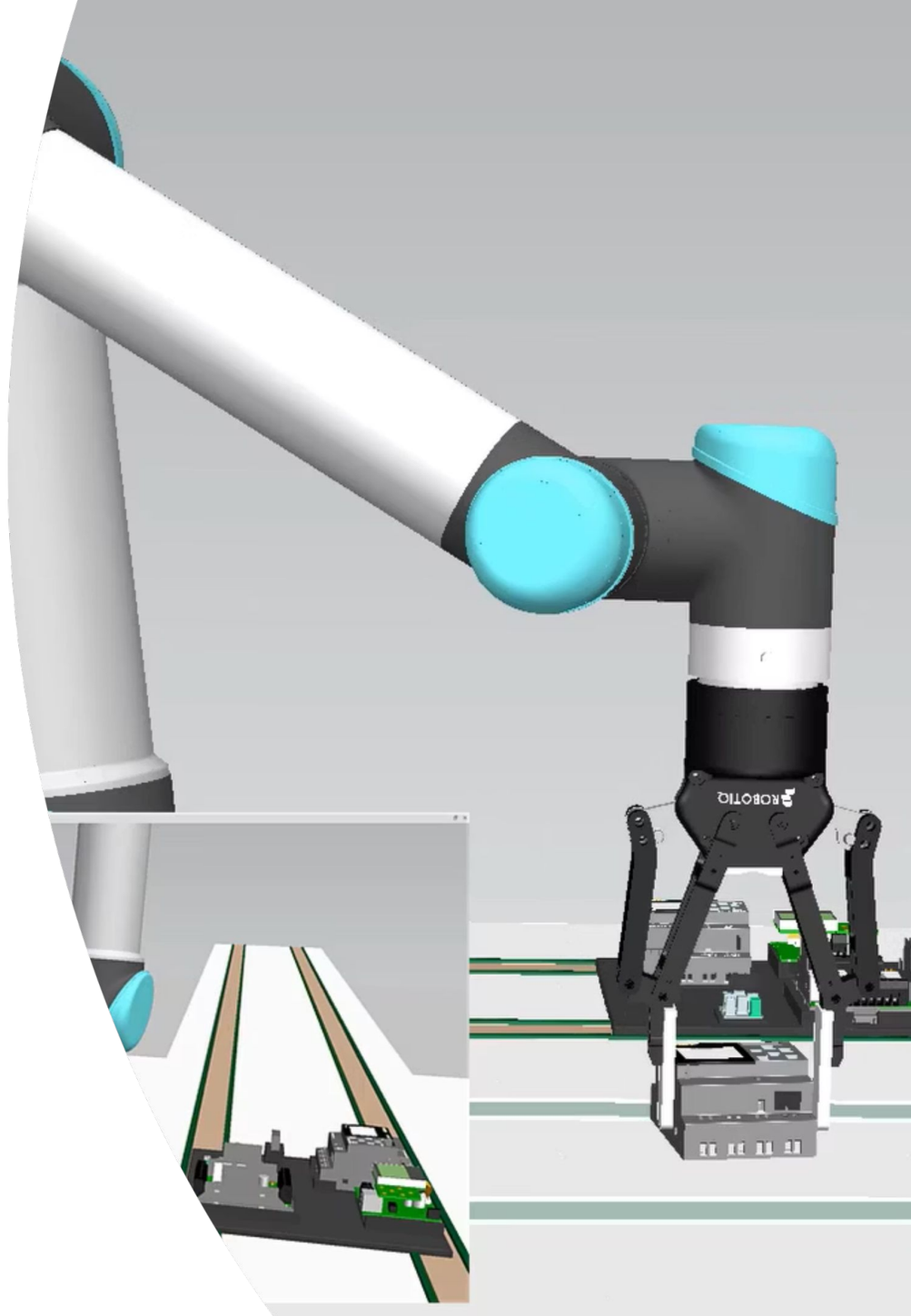
- Plotting:
  - TODO1: Update buffer and update plots.
  - TODO2: Complete multi-physics calculation.

- Path Tracing:
  - TODO3: Maintain segments buffer.
  - TODO4: Complete the color mapping function.



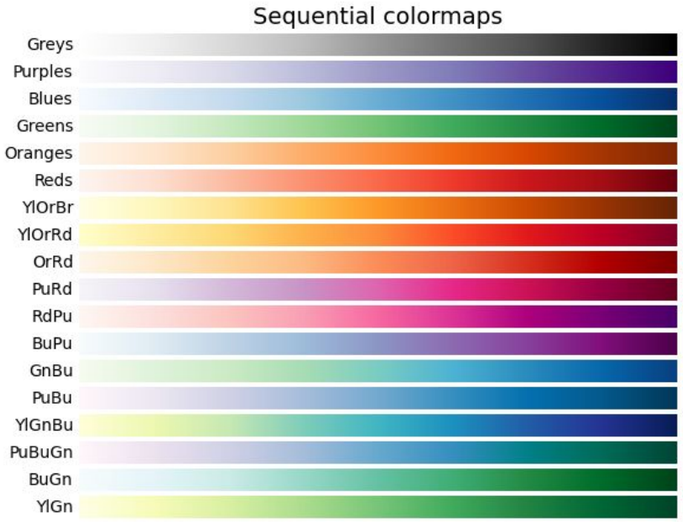


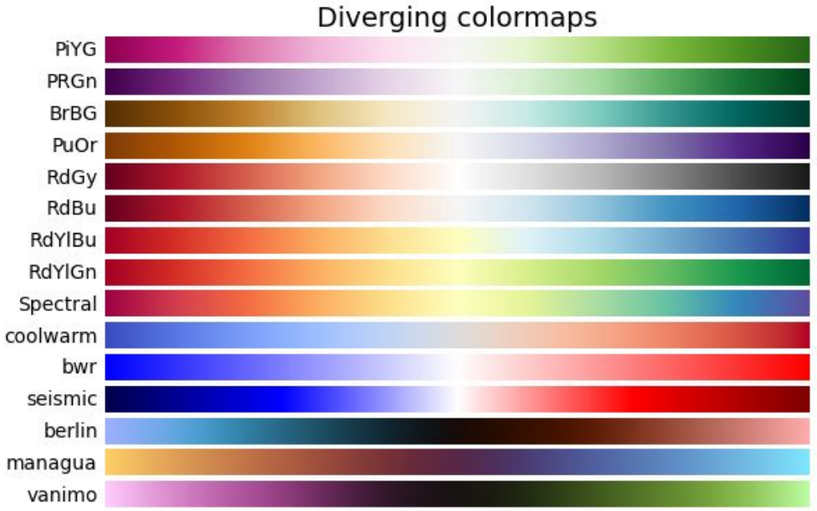
Appendix

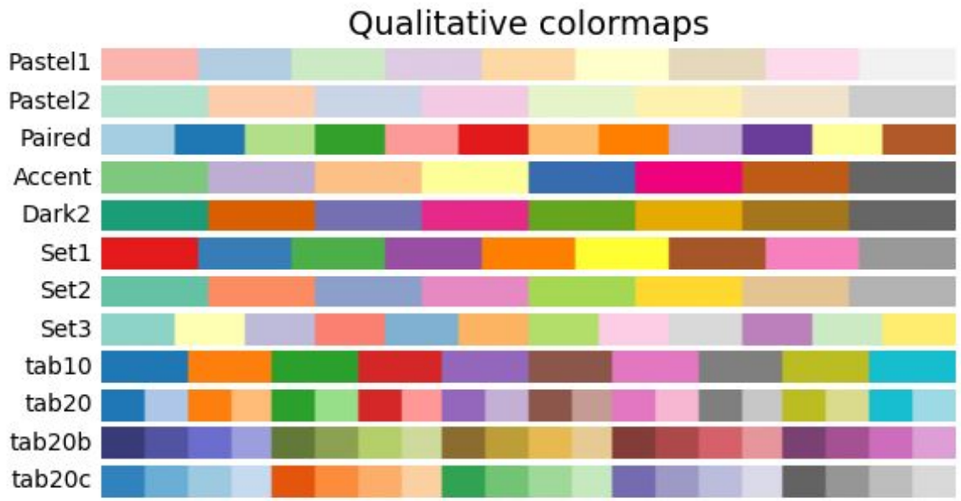


Matplotlib Built-in Colormap

- Matplotlib provides many built-in colormaps, users do not need to implement colormap manually.
- Reference: https://matplotlib.org/stable/users/explain/colors/colormaps.html

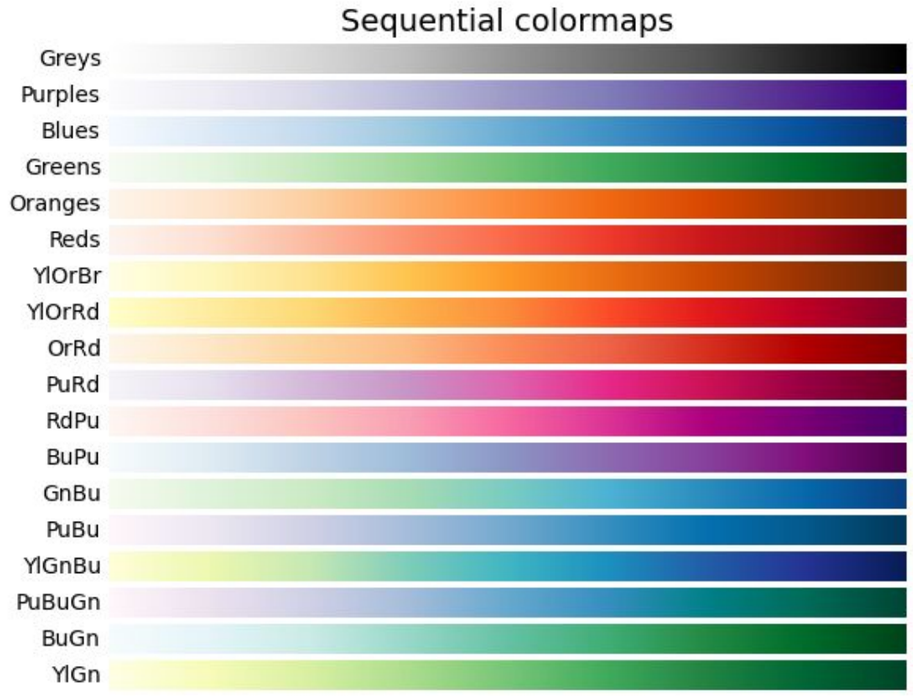


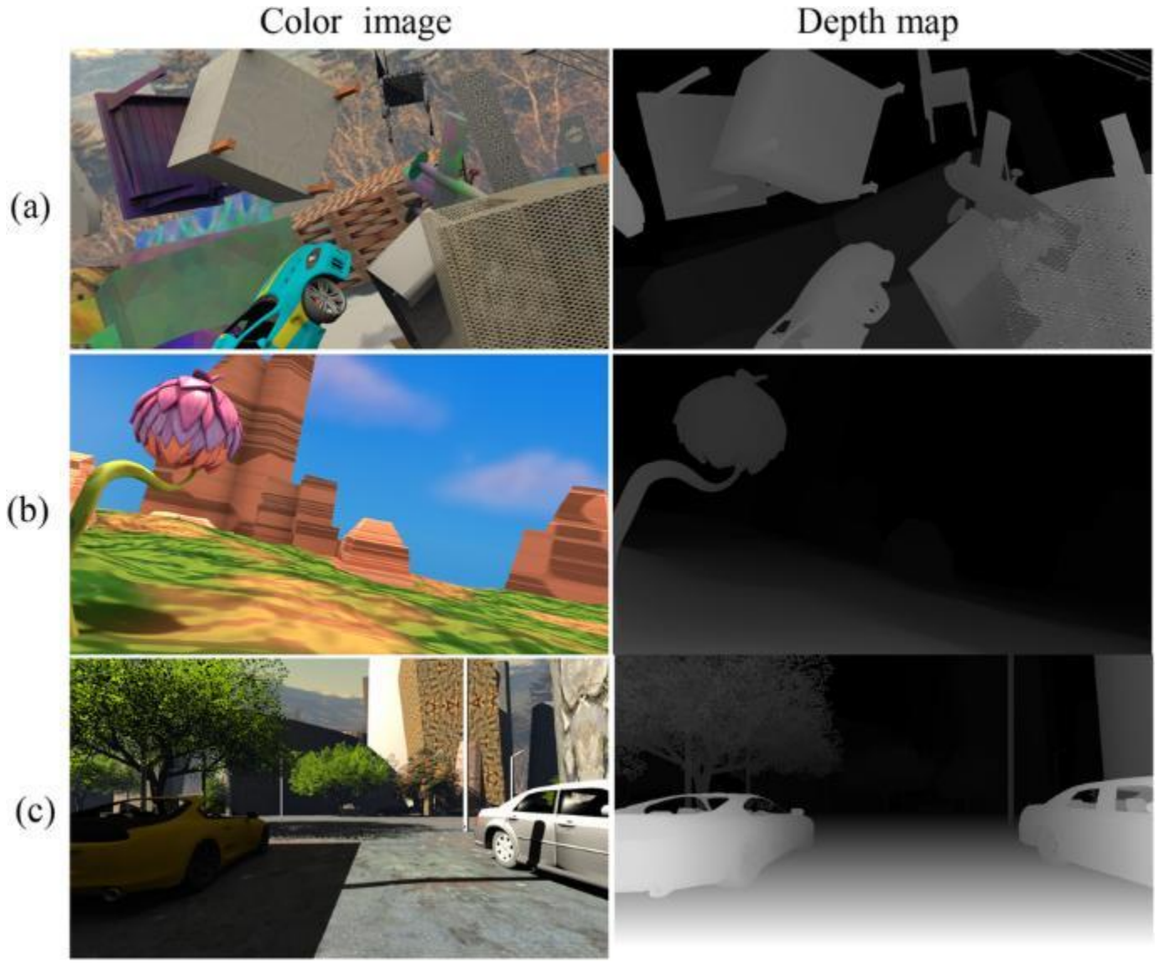




Matplotlib Built-in Colormap (Sequential)

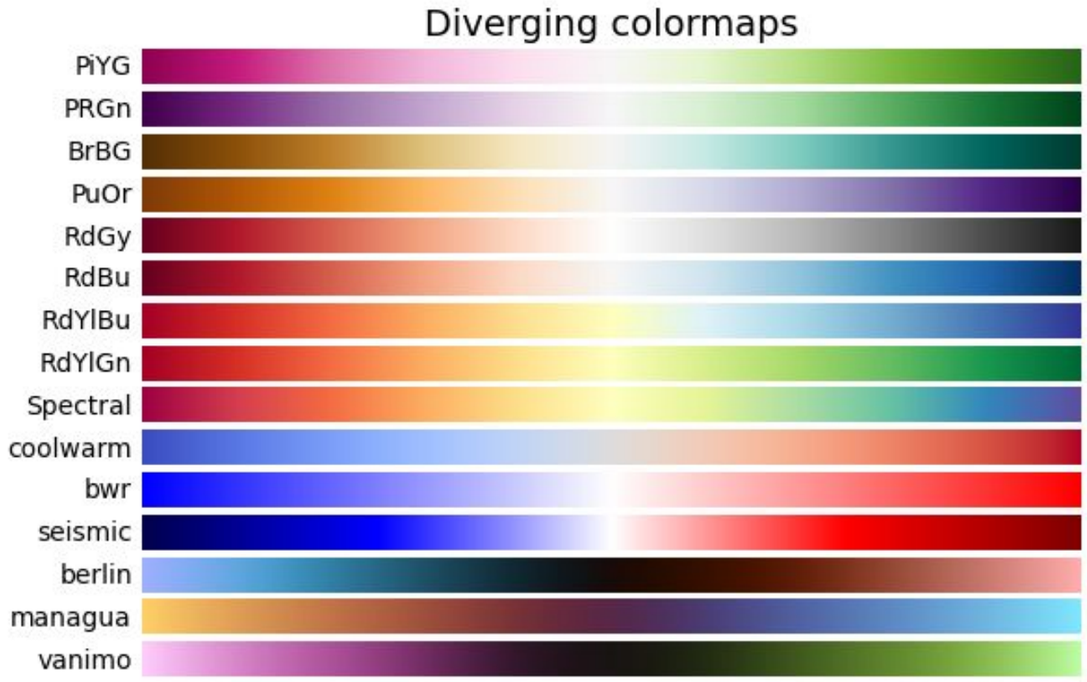
- Data format: Continuous numeric data, Ordered values (low □ high).
- Examples:
  - Temperature distribution
  - Depth map (near □ far)

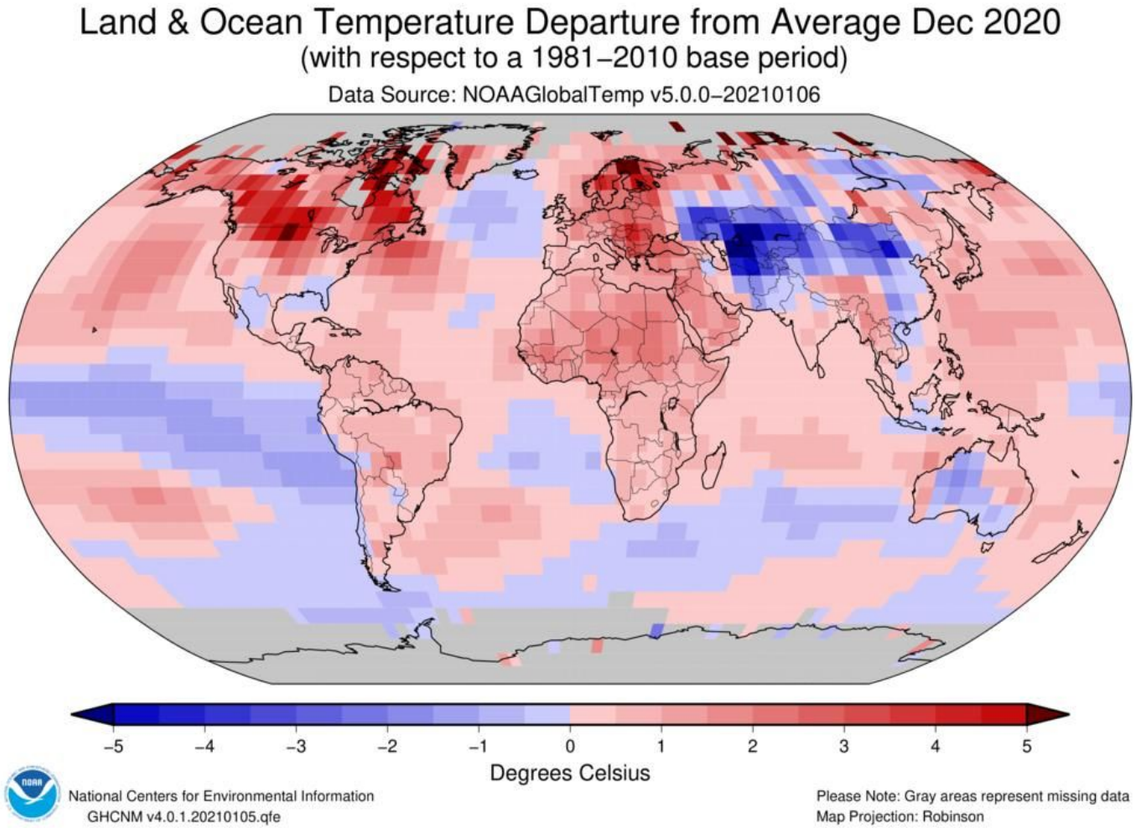




Matplotlib Built-in Colormap (Diverging)

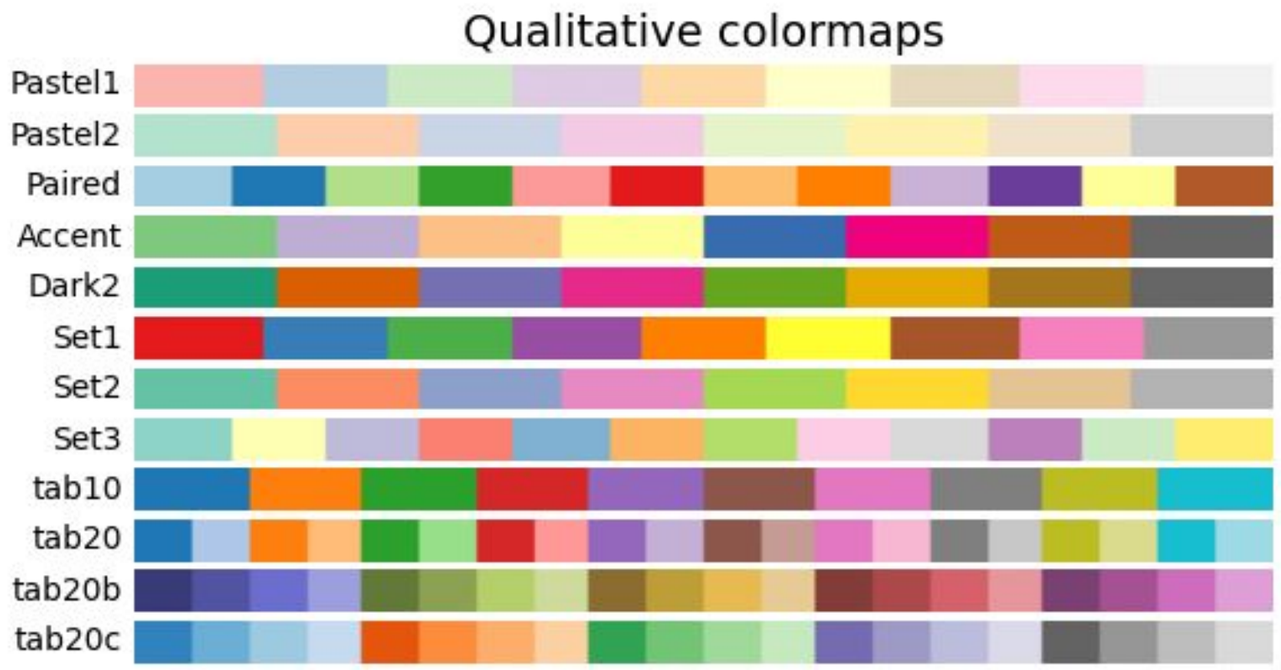
- Data format: Continuous data with a meaningful center (e.g., 0, mean).
- Examples:
  - Error map
  - Difference from average

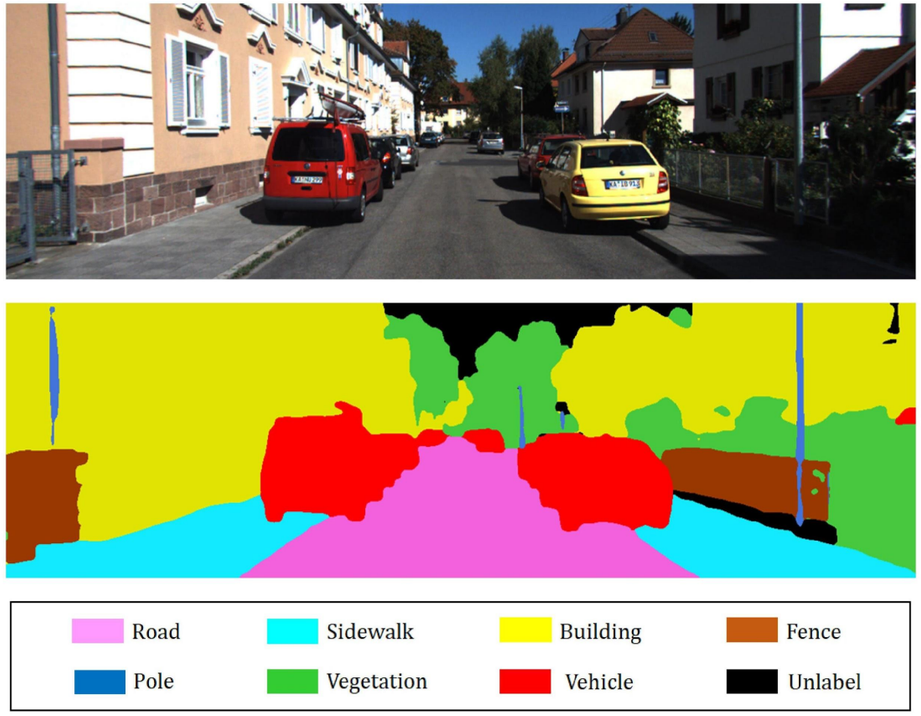




Matplotlib Built-in Colormap (Qualitative)

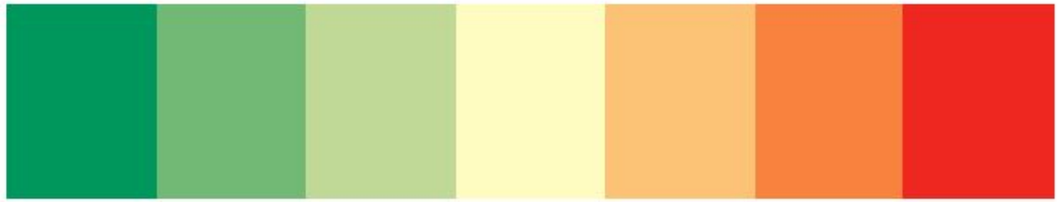
- Data format: Categorical, no inherent ordering data.
- Examples:
  - Object classes
  - Segmentation maps





Colormap Package on Github

- A Python library that provides a **collection of colormaps and color palettes** for visualization.
- Reference: https://github.com/pratiman-91/colormaps



drought_severity

ice

ice.discrete(10)